

SAMUEL OLIVEIRA LOPES

ANÁLISE COMPARATIVA DE ARQUITETURAS E PARADIGMAS NO DESENVOLVIMENTO DE JOGOS:

Um Estudo de Caso com Tetris em Múltiplas formas

Relatório Técnico elaborado conforme a
ABNT NBR 10719:10, apresentado ao
Instituto Federal de Educação, Ciência
e Tecnologia de São Paulo, como parte
dos requisitos para a obtenção do grau
de Bacharel em Ciência da Computação.

Área de Concentração: Desenvol-
vimento de Jogos

Orientador: Prof. Dr. David Buzatto

SÃO JOÃO DA BOA VISTA

2026

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO - CÂMPUS SÃO JOÃO DA BOA VISTA	Junho	2026
Bacharelado em Ciência da Computação Análise Comparativa de Arquiteturas e Paradigmas no Desenvolvimento de Jogos: Um Estudo de Caso com Tetris em Múltiplas formas Samuel Oliveira Lopes e Prof. Dr. David Buzatto		
Palavras-chave: desenvolvimento de jogos; paradigmas; comparativo; tetris; arquitetura;java; C++; python; godot;		74 páginas

Folha destinada à inclusão da Catalogação na Fonte - Ficha Catalográfica (a ser solicitada à Biblioteca IFSP – Câmpus São João da Boa Vista e posteriormente impressa no verso da Folha de Rosto (folha anterior).

Catalogação na Fonte preparada pela Biblioteca Comunitária “Wolgran Junqueira Ferreira”
do IFSP – Câmpus São João da Boa Vista

Dados da ficha

ATA N.º - DAE-SBV/DRG/SBV/IFSP

Ata de Defesa de Trabalho de Conclusão de Curso - Graduação

Na presente data realizou-se a sessão pública de defesa do Trabalho de Conclusão de Curso intitulado
apresentado(a) pelo(a) aluno(a)
do
(Câmpus Câmpus São João da
Boa Vista). Os trabalhos foram iniciados às pelo(a) Professor(a) presidente da banca examinadora, constituída pelos
seguintes membros:

Membros	Aprovação
(Presidente/Orientador)	
(Examinador 1)	
(Examinador 2)	

Observações:

A banca examinadora, tendo terminado a apresentação do conteúdo da monografia, passou à arguição do(a) candidato(a). Em seguida, os examinadores reuniram-se para avaliação e deram o parecer final sobre o trabalho apresentado pelo(a) aluno(a), tendo sido atribuído o seguinte resultado:

Aprovado(a) Reprovado(a)

Proclamados os resultados pelo presidente da banca examinadora, foram encerrados os trabalhos e, para constar, eu lavrei a presente ata que assino juntamente com os demais membros da banca examinadora.

Câmpus São João da Boa Vista,

Avaliador externo: Sim Não

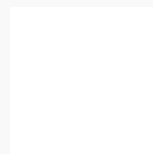
Assinatura:

Documento assinado eletronicamente por:

- PROFESSOR ENS BASICO TECN TECNOLOGICO, em
- PROFESSOR ENS BASICO TECN TECNOLOGICO, em
- PROFESSOR ENS BASICO TECN TECNOLOGICO, em

Este documento foi emitido pelo SUAP em Para comprovar sua autenticidade, faça a leitura do QRCode ao lado ou acesse
<https://suap.ifsp.edu.br/autenticar-documento/> e forneça os dados abaixo:

Código Verificador:
Código de Autenticação:



ATA N.º - DAE-SBV/DRG/SBV/IFSP

SUMÁRIO

1	INTRODUÇÃO	8
1.1	Objetivos	8
1.1.1	Objetivo Geral	8
1.1.2	Objetivos Específicos	8
2	CONSIDERAÇÕES GERAIS	9
2.1	O que é um Jogo?	9
2.2	Tipos de Jogos Tradicionais	10
2.2.1	Jogos de Cartas	10
2.2.2	Jogos de Tabuleiro	10
2.2.3	Jogos de Atlético	11
2.3	Tipos de Jogos Eletrônicos	12
2.3.1	Ação	13
2.3.1.1	FPS ou <i>First Person Shooter</i>	13
2.3.1.2	Luta	14
2.3.1.3	Plataforma	15
2.3.2	Aventura	16
2.3.3	Ação-Aventura	16
2.3.4	<i>Role Playing Game</i> ou RPG	17
2.3.5	Simulação	18
2.3.6	Estratégia	18
2.3.7	<i>Puzzle</i> ou Quebra-Cabeça	19
2.4	Tecnologias para Desenvolvimento de jogos	20
2.4.1	<i>Godot Engine</i>	20
2.4.2	Java	21
2.4.2.1	LibGDX	25
2.4.3	C++	26
2.4.3.1	Raylib	29
2.4.4	Python	30
2.4.4.1	Pygame	32
2.5	Trabalhos Correlatos	32
2.5.1	Desenvolvimento de jogos eletrônicos	32
2.5.2	Estudo comparativo entre engines de desenvolvimento de jogos 2D	33
2.5.3	Estudo comparativo de motores de jogos no desenvolvimento de um jogo 2D para web	35

3	METODOLOGIA	37
3.0.1	Estudos das Tecnologias	37
3.0.2	Implementação dos <i>Tetris</i>	38
3.0.3	Análise comparativas	39
3.0.4	Resultado das análise	39
4	ANÁLISE DOS RESULTADOS	40
4.1	Estudos das Tecnologias e Implementação	40
4.1.1	Godot	40
4.1.1.1	Nós e Scripts	41
4.1.1.2	Instanciação de Peças	41
4.1.1.3	Verificação e remoção de linhas	42
4.1.1.4	Temporização e Queda Automática	43
4.1.1.5	Movimentação	44
4.1.2	LibGDX	46
4.1.2.1	Classes	47
4.1.2.2	Gerar peças	47
4.1.2.3	Verificação e remoção de linhas	48
4.1.2.4	Temporização	49
4.1.2.5	Movimentação	50
4.1.3	Raylib	53
4.1.3.1	Classes	54
4.1.3.2	Gerar Peça	54
4.1.3.3	Verificação e remoção de linhas	56
4.1.3.4	Temporização e Queda Automática	57
4.1.3.5	Movimentação	58
4.1.4	Pygame	61
4.1.4.1	Gerar peças	61
4.1.4.2	Verificação e Remoção de linha	62
4.1.4.3	Temporização e Queda Automática	62
4.1.4.4	Movimentação	63
4.2	Resultados/Impactos	65
4.2.1	Análise Comparativa	70
4.2.1.1	Facilidade de Desenvolvimento	70
4.2.1.2	Arquitetura e Modularização	70
4.2.1.3	Curva de Aprendizado	70
5	CONCLUSÕES	72
	REFERÊNCIAS	73

RESUMO

Neste trabalho é apresentado um teste de hipótese, uma análise para descobrir quais os impactos na escolha da arquitetura, paradigmas e a linguagem para desenvolver um jogo. Utilizando *Tetris* como estudo de caso e selecionando 4 tecnologias amplamente utilizadas (*Godot*, Java com a biblioteca LibGDX, C++ com a biblioteca Raylib e Python com Pygame), observando em cada uma das tecnologias a facilidade de aprendizado, desempenho e estrutura de desenvolvimento. A metodologia se baseia em uma abordagem prática, aplicando recursos de cada tecnologia de acordo com que cada jogo é implementado, utilizando documentações oficiais e materias complementares, o estudo busca identificar as principais vantagens e limitações de cada tecnologia.

Palavras-chave: desenvolvimento de jogos; paradigmas; comparativo; tetris; arquitetura; java; C++; python; godot;

1 INTRODUÇÃO

Os jogos eletrônicos compreendem um dos setores mais relevantes da indústria do entretenimento, movimentando bilhões de dólares e engajando milhões de jogadores ao redor do mundo, a indústria de jogos eletrônicos gera mais que a indústria musical e do cinema somadas, segundo o newzoo cerca de 183 bilhões foram movimentados no mercado de jogos eletrônicos para consoles, computadores e celulares (Newzoo, 2025; Statista, 2025), além de sua importância cultural e econômica, os jogos também são um ótimo fator para estudos acadêmicos, inclusive no estudo de desenvolvimento de software, aprender lógica de programação, uso de diferentes tecnologias e à aplicação de distintos paradigmas de programação. Fazer essa comparação das escolhas torna-se importante não apenas para a prática acadêmica, mas também para orientar iniciantes e profissionais na área de desenvolvimento de jogos e descobrir qual o impacto da escolha da arquitetura e do paradigma na produtividade do desenvolvedor e na performance final de um jogo.

Com base nessas informações, será feito um teste de hipóteses analisando e comparando diferentes tecnologias de desenvolvimento de software, utilizando o jogo *Tetris* para ser feita essa análise, foi escolhido *Tetris* devido a sua estrutura ser simples e trivial.

1.1 Objetivos

1.1.1 Objetivo Geral

Comparar quatro formas diferentes de implementação do jogo *Tetris*, por meio das tecnologias *Godot*, Java, C++ e Python.

1.1.2 Objetivos Específicos

- Desenvolver Tetris no motor Gráfico *Godot*;
- Desenvolver Tetris com a linguagem Java utilizando a biblioteca LibGDX;
- Desenvolver Tetris com a linguagem c++ utilizando a biblioteca Raylib;
- Desenvolver Tetris com a linguagem Python utilizando a biblioteca PyGame;
- Coletar dados de tempo de desenvolvimento e performance dos jogos de Tetris;
- Comparar dados de tempo de desenvolvimento e performance dos jogos de Tetris;

2 CONSIDERAÇÕES GERAIS

2.1 O que é um Jogo?

Criar e participar de jogos desde os períodos mais antigos da história é um impulso básico do Homo sapiens, desde nossos ancestrais mais remotos, passando por civilizações como os gregos e os romanos, até os dias atuais, sempre existiram sistemas e práticas voltados ao ato de jogar, tendo como finalidade o entretenimento, a competição e a educação (Egenfeldt-Nielsen; Smith; Tosca, 2020).

Um jogo pode ser considerado uma atividade de ocupação voluntária, realizada dentro de certas limitações de tempo e espaço, seguindo regras determinadas e obrigatórias, com o objetivo de alcançar uma meta específica, gerando sensações de tensão, desafio e alegria (Vasques, 2025).

O primeiro jogo eletrônico da história foi lançado em 1958, chamado Tennis for Two, seu objetivo era simular uma quadra de tênis 2D e ele foi criado exclusivamente para entretenimento, permitindo que duas pessoas jogassem presencialmente. Com o passar dos anos, os jogos foram evoluindo e passaram a ser exibidos em telas de televisão e computadores, com imagens mais dinâmicas, recursos sonoros aprimorados e maior exigência de habilidade e aprendizado por parte dos jogadores.

No livro “Understanding Video Games: The Essential Introduction” (Egenfeldt-Nielsen; Smith; Tosca, 2020), os autores explicam que existem diversas metodologias de análise do que é jogo, sendo três os principais elementos:

1. **Regras:** O game design que define a estrutura do jogo;
2. **Experiência Humana:** O grau de envolvimento e imersão do jogador;
3. **Cultura:** O contexto no qual o jogo está inserido.

Esses três elementos quando juntos em uma aplicação, definem o que é um jogo, o game design vai definir as mecânicas e regras em um contexto que se está inserido que vai auxiliar na experiência humana.

De acordo com Paul Schuytema, no livro “Design de Games — Uma Abordagem Prática” (Schuytema, 2008), um jogo eletrônico é uma forma de entretenimento composta por atividades, funções e ações realizadas por um jogador, limitado por regras dentro de um universo definido, que resultam em uma condição final, o universo e as regras têm a função de dar contexto às ações do jogador, além de criar desafios e conflitos. As decisões e

ações do jogador fazem parte do processo que compõe o jogo, mas o fator mais importante não é apenas chegar ao final, e sim a experiência e o envolvimento durante o percurso.

Com base nas definições apresentadas, pode-se considerar que um jogo é algo criado pelo ser humano, composto por um conjunto de regras e limitações que estabelecem um desafio e um objetivo específico a ser alcançado, no caso de um jogo eletrônico, a definição é similar, mas com a adição de que o game design precisa ser bem estruturado, com elementos para que os jogadores se envolvam para a chegada do objetivo, tornando a experiência de jogar satisfatória, não apenas pela conquista do objetivo final, mas por todo o processo.

2.2 Tipos de Jogos Tradicionais

2.2.1 Jogos de Cartas

A categoria de jogos de cartas são aquelas no qual há a interação entre os jogadores através de um baralho de cartas, são jogos que requerem uma estratégia para serem jogadas, em torno da análise combinatoria das cartas visíveis aos jogadores ("na mesa") e das cartas que está nas mãos dos jogadores, em muitos jogos de cartas pode ter a interação e comunicação entre jogadores, permitindo o blefe, por exemplo o poker (Lucchese; Ribeiro, 2009).

Na Figura 1 é mostrada uma imagem de um baralho clássico que está presente em vários jogos de cartas:

Figura 1 – Cartas de um baralho clássico



Fonte: <<https://segredosdomundo.r7.com/dicas-de-jogos-de-baralho/>>

2.2.2 Jogos de Tabuleiro

Os jogos de tabuleiro são os que representam fisicamente um plano jogável, divisória de setores e um conjunto de peças que podem ser movidas nesse plano ao longo do decorrer

do jogo, essas peças devem ser associadas aos jogadores respectivos, em forma ou em cores, as estratégias para ganhar podem variar de jogo para jogo, assim como os objetivos, que podem ser conquistar as peças de outros jogadores, conquistar mais territórios que outros jogadores, ter mais pontos ou valores que outros jogadores (Lucchese; Ribeiro, 2009).

Na figura 2 é mostrado um exemplo de um jogo de tabuleiro, o xadrez abaixo:

Figura 2 – Tabuleiro de Xadrez



Fonte:<https://pt.wikipedia.org/wiki/Xadrez>

2.2.3 Jogos de Atlético

Jogos do tipo Atlético é uma forma tradicional de jogo, eles requerem mais habilidades físicas do que mentais, as regras de um jogo atlético requerem que o jogador execute ações precisas, que vai desde jogar uma bola em direção a um adversário, como na Queimada, ou pular de uma altura de 5 metros no salto com vara (Gularte, 2018), o uso do corpo e da resistência física é que defini para um jogador ter a habilidade de jogar um jogo atlético.

Abaixo na figura 3 está um exemplo de um jogo atlético, o basquete:

Figura 3 – Jogo atlético de Basquetebol

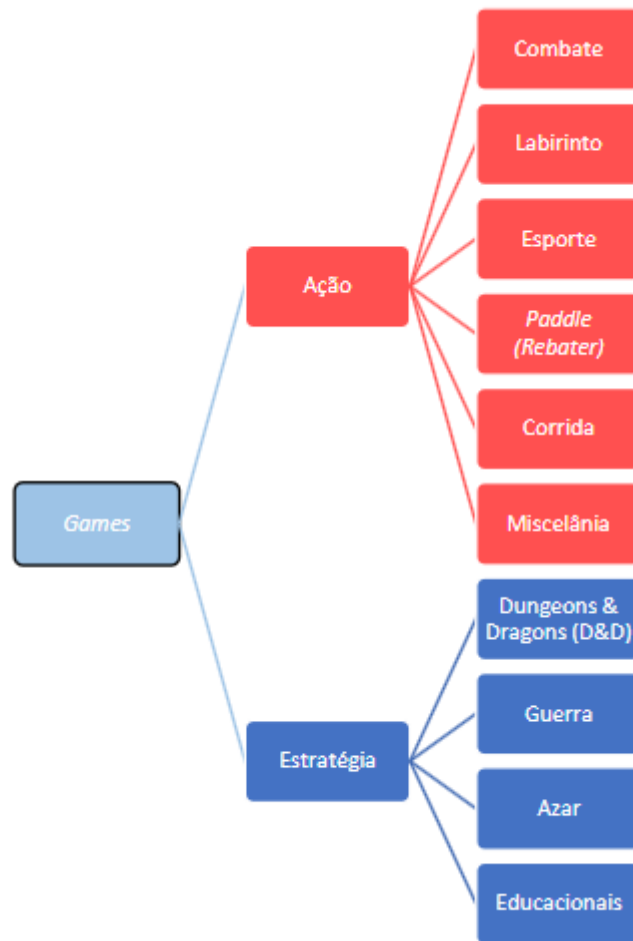


Fonte: <https://diariodonordeste.verdesmares.com.br/jogada/com-grande-atuacao-de-curry-golden-state-warriors-bate-celtics-e-conquista-nba-1.3244895>

2.3 Tipos de Jogos Eletrônicos

Nesta seção iremos discorrer a definição dos tipos, gêneros e seus subgêneros de jogos eletrônicos, Crawford (Crawford, 1982) em seu livro "The Art of Computer Game Design" categorizou os jogos eletrônicos em 2 vertentes, como na figura 4 abaixo:

Figura 4 – Classificação de Jogos segundo Crawford



Fonte: (Lucchese; Ribeiro, 2009)

Crawford já havia proposto uma taxonomia dos jogos em 1982, também é possível observar em matérias de divulgação, como no site da CodaKids (CodaKid, 2025) e da Gameopedia (Manocha, 2025) a presença de categorias com características semelhantes à de Crawford, o que consolida os gêneros e os subgêneros no senso comum dos *videogames*.

2.3.1 Ação

Um jogo de ação se caracteriza por requerer do jogador movimentos rápidos, com reflexos rápidos, com um ótimo controle visual e tempo de reações rápidos, alguns quase instantâneo (Manocha, 2025; Crawford, 1982).

2.3.1.1 FPS ou *First Person Shooter*

Um *First Person Shooter* (Jogo de Tiro em Primeira Pessoa) ou FPS ou Jogos de Tiro se defini através do jogador constantemente está mirando e atirando em alvos com a finalidade de alcançar um objetivo. Esses jogos se caracterizam por combate entre jogador e inimigo a longas distâncias (Manocha, 2025).

Um clássico dos jogos de FPS é mostrado na figura 5 abaixo:

Figura 5 – Jogo de FPS *Bioshock Infinite*



Fonte: www.imdb.com/es/title/tt1712064/mediaviewer/rm1094737152/

2.3.1.2 Luta

Em jogos de luta se caracteriza por um jogador está lutando contra uma máquina ou contra um ou mais jogadores, é muito comum em jogos de luta, utilizar as teclas que fazem o jogador bater, bloquear, pular encadear sequências de ataques em "combos". O combate corpo a corpo está sempre ligado à alguma arte marcial, que está ligado aos personagens do jogo de luta, os jogadores normalmente são representado de lado em um plano 2D, há alguns jogos que utilizam planos paralelos no plano 2D para movimentos do jogador (CodaKid, 2025; Manocha, 2025).

Um bom exemplo de um jogo de luta chamado "*Street Fighter*" é mostrado na figura 6 abaixo:

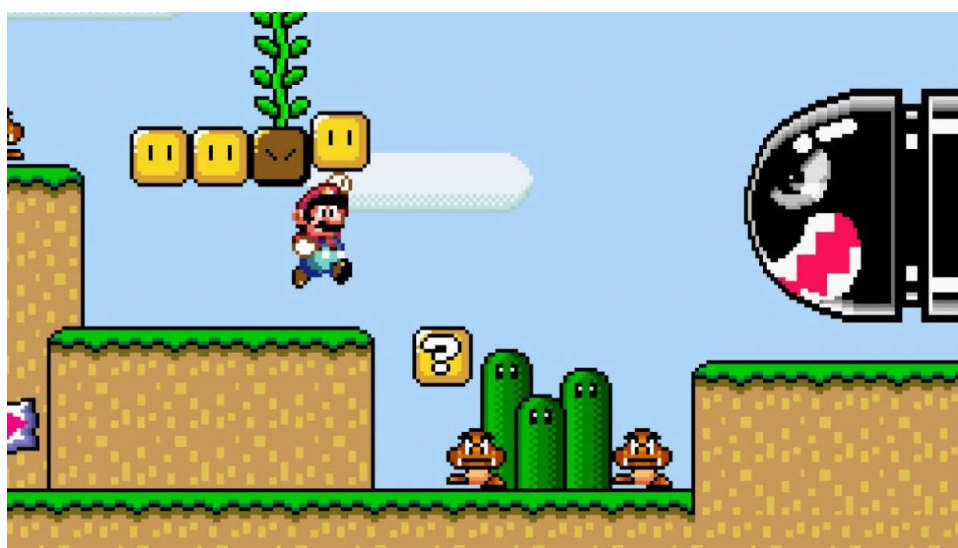
Figura 6 – Jogo de luta *Street Fighter*

Fonte: www.techtudo.com.br/noticias/2012/03/especial-a-historia-da-serie-street-fighter-parte-1.ghml

2.3.1.3 Plataforma

No subgêneros de jogos de plataforma, os jogadores tem que pular, escalar ou usar alguma outra mecânica para se mover de uma plataforma até a outra. O jogador deve aprender a lidar inimigos ou evita-los (Manocha, 2025; CodaKid, 2025).

É um subgênero muito consolidado, na década de 1980 com o lançamento de *Super Mario Bros.* e *Donkey Kong*, se tornou um subgênero muito famoso (Manocha, 2025; CodaKid, 2025). Um exemplo de jogo de plataforma é mostrado na figura 7 abaixo:

Figura 7 – Jogo de Plataforma popular *Super Mario World*

Fonte: <https://xboxplay.games/uploadStream/679.jpg>

2.3.2 Aventura

Jogos de aventura se caracterizam-se por oferecer ao jogador a possibilidade de se mover em um cenário ou mundo complexo, acompanhado de um enredo estruturado. Ao longo da aventura, o jogador adquire ferramentas e/ou itens e utiliza em obstáculos encontrados pelo caminho, resolve quebra cabeças, explora o cenário, até atingir um objetivo final ou até chegar ao desfecho da narrativa (Crawford, 1982; CodaKid, 2025), uma aventura combina elementos de exploração e narrativa de enredo para serem imersivos ao jogador. (Manocha, 2025)

Um bom exemplo de jogos de aventura é a franquia de jogos "The legend of Zelda", abaixo é mostrado na figura 8 a imagem de um dos jogos dessa franquia:

Figura 8 – Jogo de Aventura *The Legend of Zelda: Ocarina Of Time*



Fonte: en.wikipedia.org/wiki/ThelegendofZelda

2.3.3 Ação-Aventura

Jogos de Ação-Aventura se definem pela combinação de elementos de ação e de aventura, são bem similares a ambos os gêneros, geralmente oferecem ao jogador uma narrativa construída ao longo de elementos de ação, o avanço do enredo altamente depende do movimento do personagem do jogador, que desencadeia os eventos da história e afeta o fluxo do jogo (CodaKid, 2025).

Um exemplo de jogo de gênero de ação-aventura é mostrado na figura 9 abaixo:

Figura 9 – Jogo de ação-aventura *Tomb Raider*

Fonte: <https://criticalhits.com.br/reviews/tomb-raider-review/>

2.3.4 Role Playing Game ou RPG

Em um RPG ou Jogo de interpretação de Papeis, o jogador assume um papel de um personagem, criando a aparência e evoluindo-o em experiência e força durante o fluxo do jogo (CodaKid, 2025), um RPG possibilita o jogador jogar no papel escolhido por ele, por exemplo: em um RPG de fantasia, o jogador pode jogar de mago ou de arqueiro, junto de uma narrativa, os jogadores fazem progresso, desbloqueiam habilidades, fazer progresso em um RPG, geralmente está amarrado a completar missões que está relacionado ao enredo principal e a completar missões secundárias, que não está relacionado ao enredo principal (Manocha, 2025).

Um exemplo de um jogo de RPG é mostrado na figura 10 abaixo:

Figura 10 – Jogo de RPG *The Elders Scrolls: Morrowind*

Fonte: <https://www.ign.com/articles/2002/06/17/elder-scrolls-iii-morrowind-review>

2.3.5 Simulação

Um jogo de simulação tende a simular ou imitar eventos da vida real na forma mais realista possível (Manocha, 2025), a simulação nesses jogos deve assumir diferentes finalidades como educar o jogador, permitir análises e previsões, ou simplesmente oferecer entretenimento. em geral não há um objetivo final delimitado para chegar em uma simulação, mas proporciona ao jogador a liberdade para controlar um personagem ou o ambiente (CodaKid, 2025).

Em um jogo de simulação, o jogador precisa estar familiarizado com elementos próprios da simulação, por exemplo, em *SimCity*, é necessário compreender conceitos relacionados à gestão urbana, como as funções de um prefeito e os processos de construção e organização de uma cidade, é mostrado na figura 11 uma imagem do jogo abaixo:

Figura 11 – Jogo de simulação *Sim City 4*



Fonte: <https://www.lifewire.com/simcity-4-starting-new-city-840147>

2.3.6 Estratégia

Um jogo de estratégia requerem que o jogador pense em planejamento táticos e lógicos tendo como finalidade o objetivo de ganhar (Manocha, 2025), o jogador deve realizar uma série de jogadas em turnos contra um ou mais oponentes. Por exemplo, em *StarCraft* o jogador deve reunir e distribuir recursos, posicionar tropas nas áreas certas e fornecer a elas os recursos necessários para vencer com eficiência, exigindo pensamento táticos e lógica de estratégia(CodaKid, 2025; Manocha, 2025).

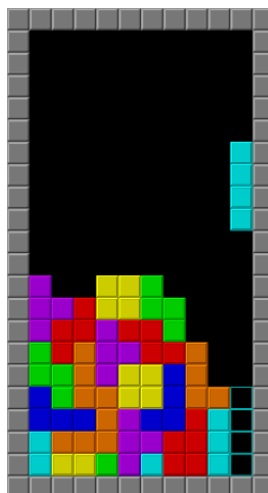
Um exemplo de um jogo de estratégia é representado na figura 12 abaixo:

Figura 12 – Jogo de estratégia *Into The Breach*

Fonte: <https://www.pcgamer.com/into-the-breach-review/>

2.3.7 *Puzzle* ou Quebra-Cabeça

Jogos do tipo *Puzzle* ou quebra-cabeças tem como aspecto central: o jogador resolve problemas utilizando a lógica, pode haver mais de uma solução em um quebra-cabeça ou em um enigma e um jogador não deve achar a solução aleatoriamente, para resolver, o jogador deve reconhecer padrões, descobrir pistas ou até realizar certos movimentos em certas ordens para resolver o quebra-cabeça ou o enigma (Manocha, 2025). Jogos de quebra-cabeça como *Portal* (2007), exigem que o jogador utilize mecânicas de física e manipular portais para passar obstáculos (Manocha, 2025) e no jogo *Tetris*, exige do jogador organizar peças geométricas em tempo real, testando a agilidade mental e a coordenação espacial, é mostrado um exemplo na figura 13 abaixo:

Figura 13 – Jogo de quebra-cabeça *Tetris*

Fonte: wikipedia.org/wiki/Tetris

2.4 Tecnologias para Desenvolvimento de jogos

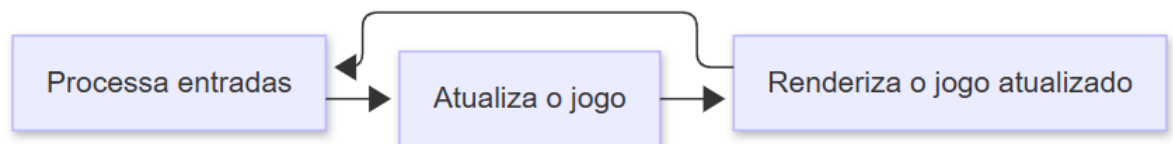
Para desenvolver um jogo, é necessário conhecimentos de várias áreas distintas como: música, programação de algoritmos, desenhos, gráficos e outros. A utilização dessas áreas de forma individual, seria extremamente complexa, por isso são criados *engines* ou motores gráficos (Alves, 2020).

Uma *engine* abstrai esses elementos que precisaria de um conhecimento muito específico e extensos, essa abstração não é algo padronizado, alguns motores gráficos tem mais abstração, outros possui menos abstração (Alves, 2020).

Motores gráficos são ambientes para desenvolvimento de jogos, como uma IDE¹, gerenciando o *loop* e possuindo módulos para a criação de diferentes tipos de jogos, o *looping* de um jogo é um processo fundamental de um jogo, defini o ciclo de vida de um jogo (Alves, 2020).

Apesar da estrutura de um *loop* ser muito complexo e detalhado, todas as *engines* seguem o mesmo padrão. Essa estrutura de *looping* passa por uma entrada, a partir da entrada, atualiza as estruturas dentro do jogo, e por último renderiza o jogo no estado atual e assim é feito a repetição desse processo (Alves, 2020), como mostrado na figura 14 abaixo:

Figura 14 – Ciclo de vida de um jogo



Fonte: Adaptado de (Alves, 2020)

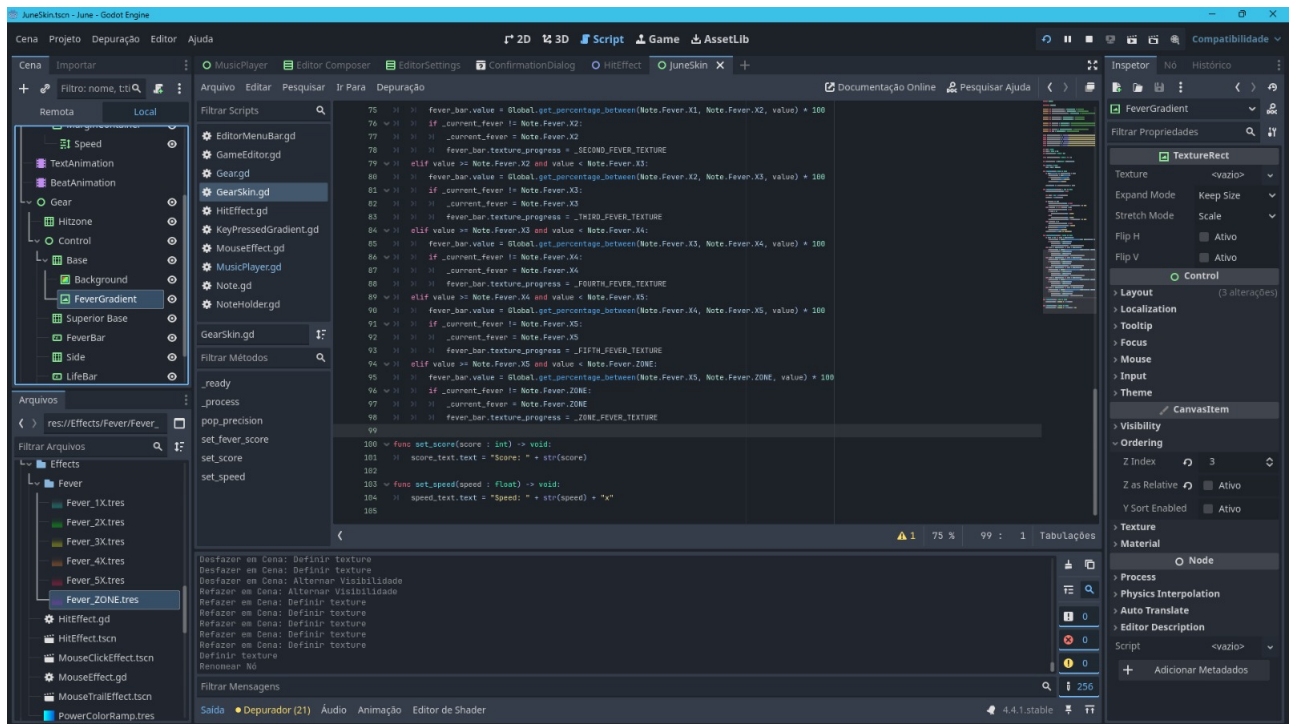
2.4.1 Godot Engine

A *Godot Engine* é uma ferramenta grátis e de código aberto licenciado pelo MIT para criação de jogos 2D e 3D (Godot Engine Community, 2025), a *Godot Engine* oferece ao desenvolvedor ferramentas para criação de jogos, gerenciamento de *assets* e também oferece um editor de script integrado, o desenvolvimento na *Godot Engine* é baseado em

¹ Integrated Development Environment (Ambiente de Desenvolvimento Integrado)

classes de objetos chamados nós (nodes) (Salmela, 2022; Godot Engine Community, 2025). Existem muitos tipo de nós disponíveis, nós para gráficos, física, interface de usuário, audio e de outros tipos, esses nós são organizados em uma cena conforme o desenvolvedor quiser para comportamentos mais complexos (Salmela, 2022; Godot Engine Community, 2025), assim como um script pode ser anexado a um nó para também conseguir um comportamento específico. *Godot Engine* tem uma linguagem para programar scripts chamada GDScript, a sintaxe é bem similar ao Python por causa da indentação mas não é baseado na mesma, também uporta a linguagem C# (Godot Engine Community, 2025). Um exemplo do ambiente de desenvolvimento na *Godot Engine* é mostrado na figura 15 abaixo:

Figura 15 – Janela do editor Godot durante o desenvolvimento do jogo



Fonte: Elaborado pelo Autor

Godot Engine possui fluxos separados de renderização 2D e 3D, na renderização 3D oferece físicas de sombra e luz, já na renderização em 2D também oferece luz, sombra e máscaras. Em ambos os fluxos de renderização possibilita o desenvolvedor utilizar físicas para simulação e *ray casting*² (Godot Engine Community, 2025).

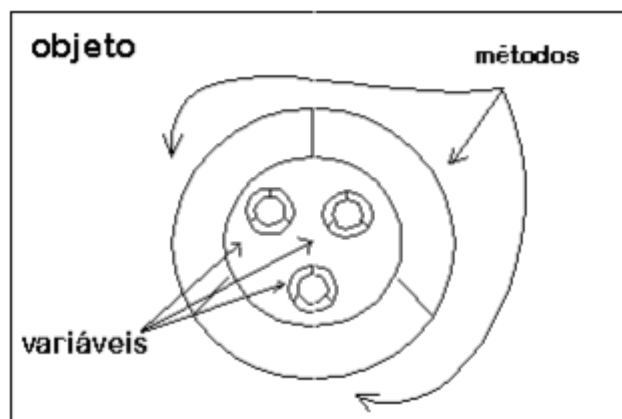
2.4.2 Java

Java é uma linguagem de programação de baixo nível que surgiu na década de 90, como uma necessidade para tráfegar dados na web, a linguagem é baseada no paradigma de orientação à objeto, é possível encapsular dados dentro de um bloco e criar métodos

² Técnica de renderização 3D a partir de um ponto de observação

de manipulação desses dados. Um exemplo de encapsulamento é mostrado na figura 16 abaixo:

Figura 16 – Encapsulamento de variáveis e métodos



Fonte: (Indrusiak, 1996)

Um exemplo em código de encapsulamento:

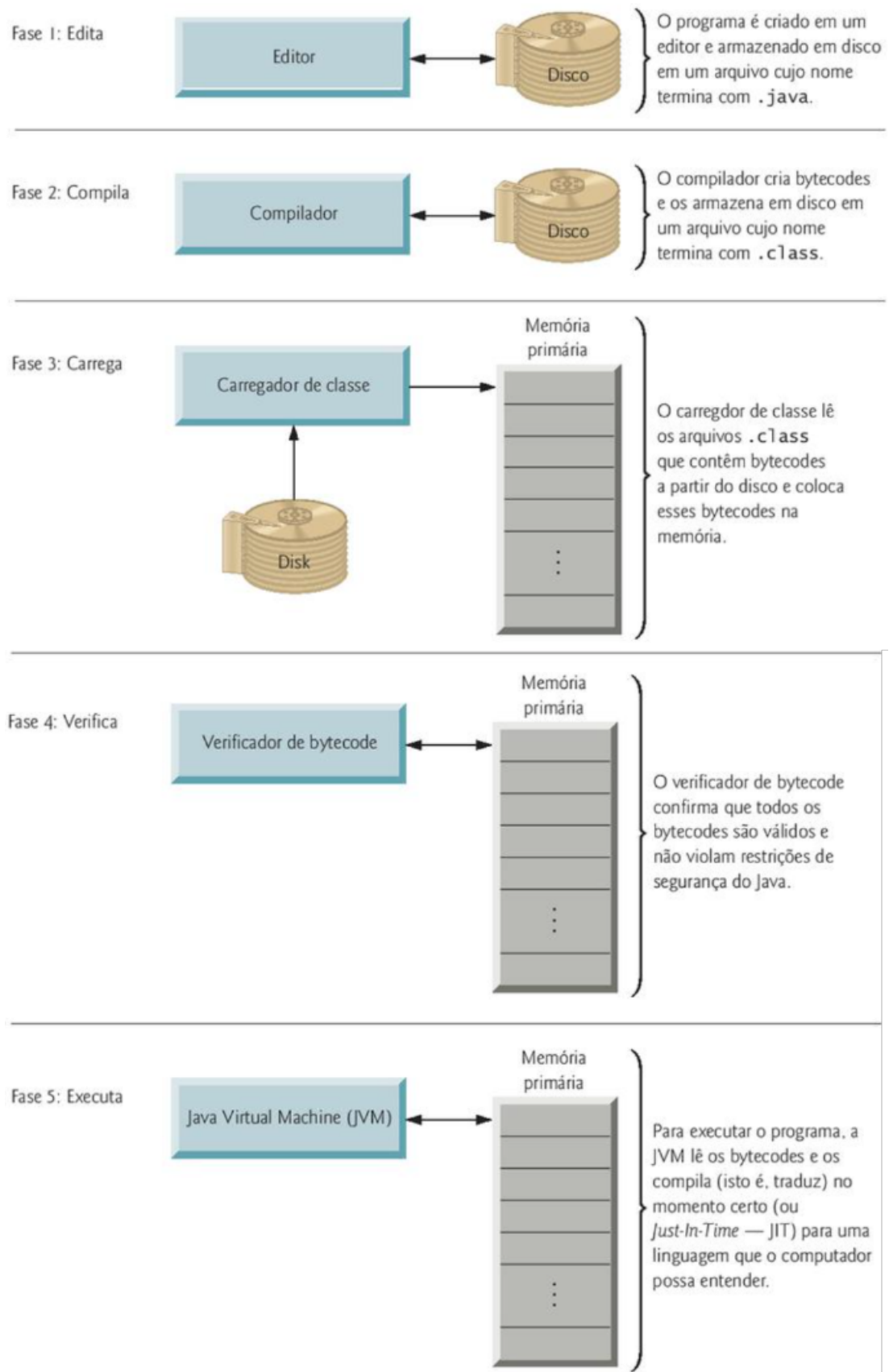
```
1 public class Classe{
2
3     String variavel;
4     int num;
5
6
7     public String getVariavel(){
8         return variavel;
9     }
10
11     public void setVariavel(String variavel){
12         this.variavel = variavel;
13     }
14
15     public int getNum(){
16         return num;
17     }
18
19     public void setNum(){
20         this.num = num;
21     }
22 }
```

A linguagem também permite a modularização das aplicações, reuso e manutenção simples do código implementado (Indrusiak, 1996). Sob as camadas internas, Java em seus recursos fundamentais de uma linguagem, como estruturas de controles, estruturas de definição, tratamentos de exceções, construtores de objeto, modificadores de acesso, tipos de dados e outros, é padronizado e mantém um conjunto de recursos limitado comparado

á outras linguagens, essa padronização e simplicidade se mostra na ausência no uso de ponteiros e gerenciamento de memória por meio da programação, fazendo com que a programação fique mais eficiente (Indrusiak, 1996).

A linguagem Java é tanto interpretada como compilada, após escrever um código em Java, é salvo o arquivo fonte e em seguida é possível compilar este arquivo fonte, produzindo um arquivo binário chamado de arquivo classe, esses arquivos não são executados diretamente pelos processadores, esses arquivos estão em um formato chamado *bytecode* e assim podem ser executados em qualquer interpretador de Java, o código precisa ser compilado apenas uma vez, pois os *bytecode* são gerados dentro do arquivo classe e podem ser executados em qualquer plataforma de hardware e software (Indrusiak, 1996; Deitel; Deitel, 2016), é mostrado mais detalhadamente na figura 17 abaixo:

Figura 17 – Como funciona a execução e compilação de um código em java



Fonte: (Deitel; Deitel, 2016)

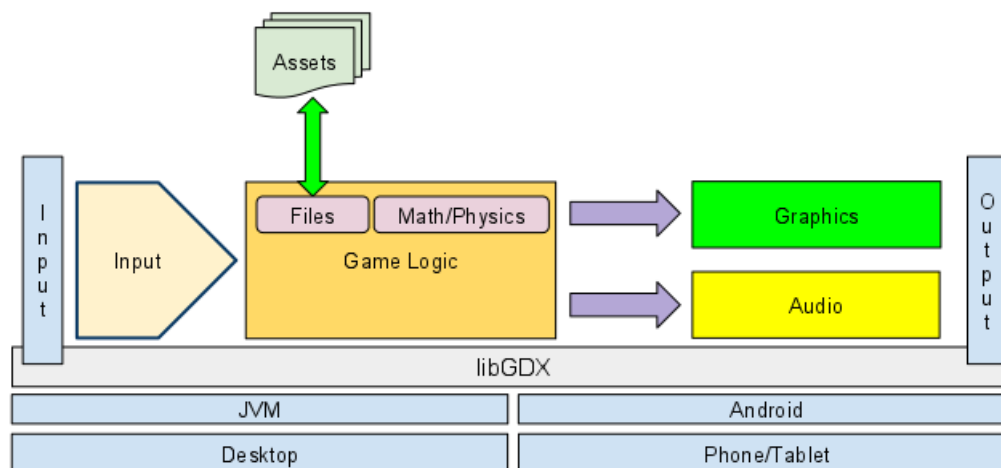
2.4.2.1 LibGDX

LibGDX é um framework³ escrito em java, utiliza OpenGL⁴ e é código aberto para desenvolvimento de jogos 3D e 2D para computadores e celulares (LibGDX, 2025), a LibGDX oferece módulos para que a arquitetura de um jogo possa ser construído, são eles:

- **Módulo de entradas:** Dá ao desenvolvedor uma forma unificada de modelos de entradas, detectando teclados, telas sensíveis ao toque, aceletometros e mouses (LibGDX, 2025).
- **Módulo de gráficos:** Proporciona o desenho de imagens na tela utilizando OpenGL ES⁵ fornecido por hardware (LibGDX, 2025).
- **Módulo de arquivos:** Concede o acesso a métodos convenientes para leitura e gravação de arquivos de mídia (LibGDX, 2025).
- **Módulo de áudio:** Disponibiliza recursos de áudio para gravação e reprodução de arquivos áudio em todas as plataformas (LibGDX, 2025).
- **Módulo de redes:** Fornece ao desenvolvedor métodos para utilizar recursos de rede, usando protocolos de internet e comunicação cliente e servidor (LibGDX, 2025).

Com esses módulos disponíveis na biblioteca LibGDX, abaixo na figura 18 mostra um diagrama como esses módulos são usados em um ciclo de vida de um jogo:

Figura 18 – Ciclo de vida de um jogo com os módulos da LibGDX



Fonte: (LibGDX, 2025)

³ Conjunto de ferramentas, bibliotecas, componentes prontos e boas práticas que fornecem uma estrutura base para o desenvolvimento de software.

⁴ Interface de programação de aplicações (API) gráfica multiplataforma que permite renderizar gráficos 2D e 3D. Disponível em: <https://www.opengl.org/>

⁵ (Graphics Library for Embedded Systems) Disponível em: <https://www.khronos.org/opengles/>

Além disso, LibGDX oferece ao desenvolvedor a possibilidade de utilizar partículas 2D, manipulação de *bitmaps* e editor de texturas, junto com isso (LibGDX, 2025; Firmansyah; Adhitya, 2021), existem métodos principais, são essenciais para manter o ciclo de vida de um jogo, são eles na tabela 1 abaixo:

Tabela 1 – Ciclo de Métodos do LibGDX

Nome do Método	Descrição
Create()	Este método será usado uma vez quando a aplicação for criada.
Resize(int width, int height)	Este método será sempre usado quando o tamanho da tela do jogo estiver mudando e não estiver em pausa. Há um parâmetro de largura e altura da tela que é atualizado sempre que o jogo for redimensionado.
Render()	Este método é usado como repetição do jogo na aplicação. A lógica do jogo é atualizada aqui.
Pause()	No Android, este método é usado quando o botão <i>home</i> é pressionado ou quando há uma chamada telefônica. No desktop, este método é usado apenas uma vez antes do método <i>dispose</i> .
Resume()	Este método é usado apenas no Android para continuar a aplicação após estar em pausa.
Dispose()	Este método é usado para fechar a aplicação.

Fonte: Adaptado de (Firmansyah; Adhitya, 2021)

2.4.3 C++

A linguagem de programação C++ de alto nível, se originou a partir da linguagem de programação C, foi criado para abstrair dados, suportar a programação orientada a objeto e suprir uma demanda de programas de softwares que ocorreu na década de 1980 (Deitel; Deitel, 2005; Stroustrup, 1986).

C++ é uma linguagem que mistura tanto a programação orientada a objeto tanto quanto a programação procedural, assim o programador tem uma maior flexibilidade escolhendo o paradigma que mais se adequa ao projeto, a criação de códigos fica mais eficiente e é mais reutilizáveis por causa da modularização⁶.

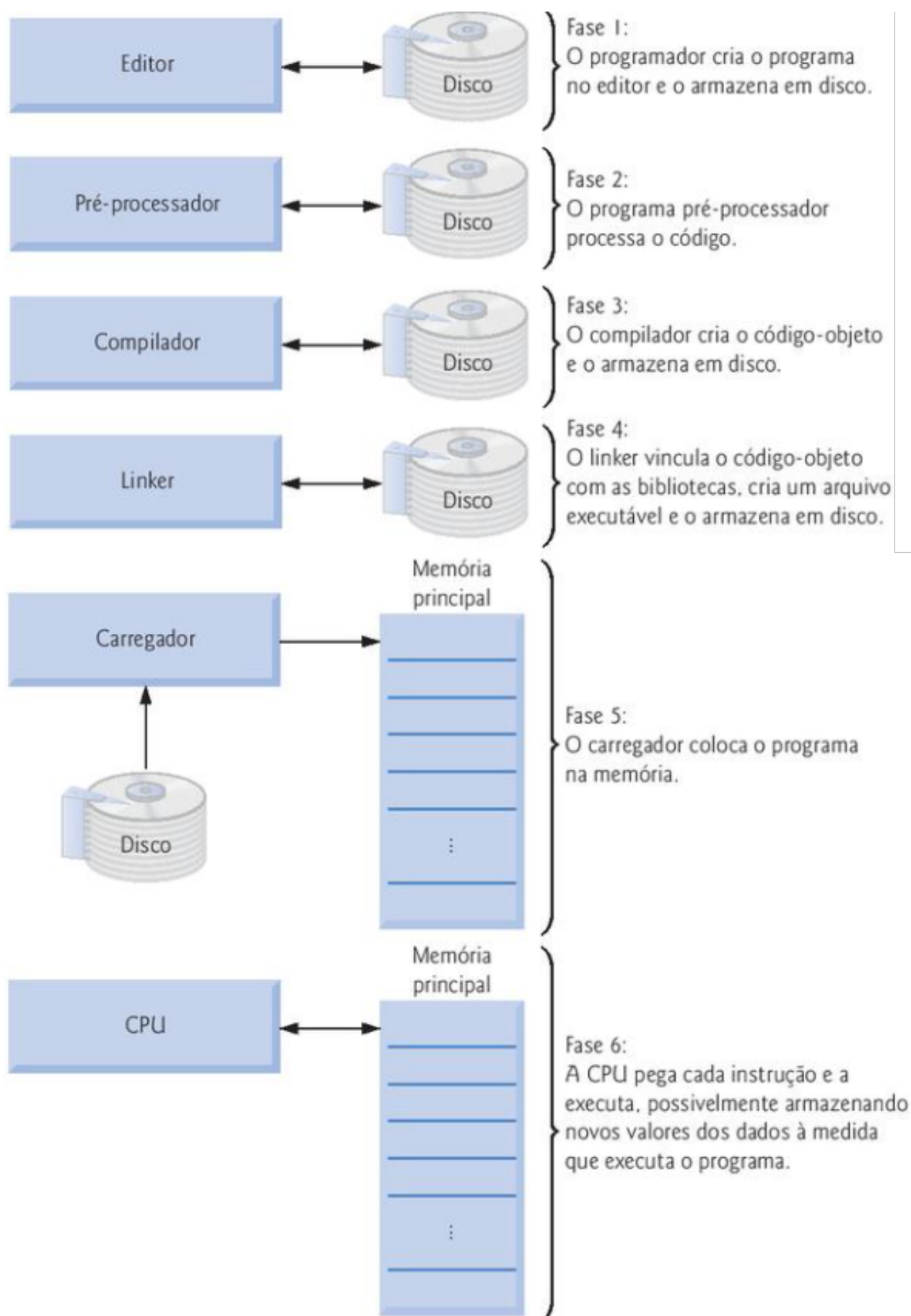
Os programas em C++ são feitos classes e funções existentes na C++ *Standart Library*, além disso a linguagem suporta a programação genérica por meio de modelos de diferentes tipos, sem a necessidade de reescrever o código (Turos, 2024; Deitel; Deitel, 2005).

Os códigos feitos em C++ ao serem executados passam por 6 fases: edição, pré-processamento, compilação, linkagem, carregamento e execução, como mostrado na figura 19

⁶ Tarefas separadas em funções ou procedimentos, cada uma realizando uma parte do programa

abaixo:

Figura 19 – As fases para rodar um código em C++



Fonte: (Deitel; Deitel, 2005)

2.4.3.1 Raylib

Raylib é uma biblioteca de programação de Jogos 2D e 3D de código aberto escrita em C. A Raylib não provê uma API de documentação igual o Godot ou a LibGDX, apenas uma lista com todas funções disponíveis e um exemplo de como usa-las, Raylib tem suporte para computadores, tanto Desktop quanto para Web e para celulares, apesar de ser escrita em C, a linguagem suporta também pode ser usada em linguagens como C++ e Python (Valladares, 2025). Assim como a LibGDX e o godot, a Raylib tem funções próprias que compõe a arquitetura de um jogo (Valladares, 2025), abaixo na tabela 2 é mostrado as funções e o que fazem:

Tabela 2 – Funções básicas para utilizar a raylib

Nome da função	Descrição
InitWindow(Width, Height, "My Game")	Este método será usado uma vez quando a aplicação for criada. Há parâmetro de largura e altura da tela e o nome da janela
SetTargetFPS(60)	Este método configura quantos quadros por segundo o jogo será renderizado
BeginDrawing()	Este método é usado dentro de uma estrutura de <i>loop</i> , serve para desenhar na janela
ClearBackground(RAYWHITE)	Este método é usado para limpar a janela em uma cor específica para evitar erros visuais
EndDrawing()	Este método é utilizado para terminar o desenho dentro da janela
CloseWindow()	Método para fechar a aplicação

Fonte: Elaborada pelo autor com base em (Valladares, 2025)

Também abaixo é mostrado um código base para a programação de um jogo em Raylib:

```

1 #include "raylib.h"
2
3 int main() {
4     // Determina a altura e largura da janela
5     const int screenWidth = 800;
6     const int screenHeight = 450;
7
8     // Inicializa a janela
9     InitWindow(screenWidth, screenHeight, "My Game");
10
11     // Configura quantos quadros por segundo
12     SetTargetFPS(60);

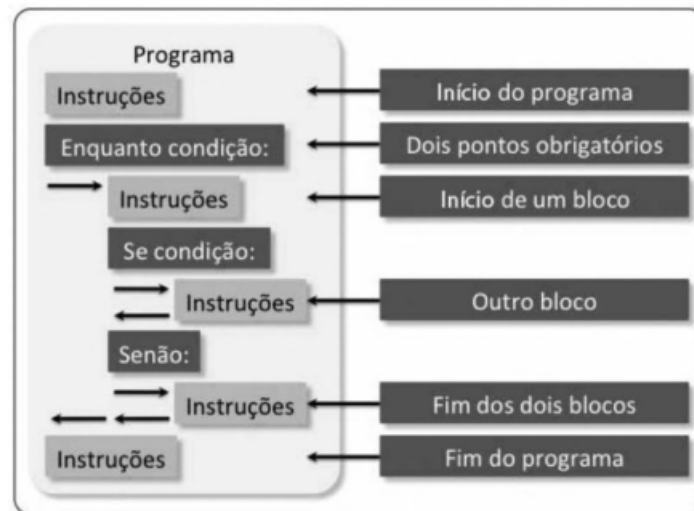
```

```
13
14     // O Loop do jogo
15     while (!WindowShouldClose() /*WindowShouldClose retorna
16     verdadeiro se esc é apertado e fecha a janela*/) {
17
18         // Configura o Canva
19         BeginDrawing();
20         // Limpa o canva para não haver erros
21         ClearBackground(RAYWHITE);
22
23         // ...
24         // Lógica do jogo é inserida aqui
25         // ...
26
27         // Fecha o canva
28         EndDrawing();
29     }
30     CloseWindow();
31     return 0;
32 }
```

2.4.4 Python

Python é uma linguagem de alto nível que surgiu na década de 1990 especializada para físicos e engenheiros, é usada popularmente nos dias atuais por empresas alta tecnologia, diferentemente de várias outras linguagens de programação populares, python o código precisa ser bem indentado pois depende do recuo para o código funcionar, o que torna a linguagem com sintaxe simples, clara e concisa(Downey, 2012; Oracle Brasil, 2025), na figura 20 abaixo é demonstrado essa indentação:

Figura 20 – Indentação de um programa comum em python



Fonte: (Borges, 2013)

Python é interpretada, ou seja, não é criado um arquivo com uma linguagem de máquina para o computador ler, o código é lido por um interpretador na máquina virtual python (Downey, 2012), como mostrado na figura 21 abaixo:

Figura 21 – Diagrama de um interpretador



Fonte: adaptado de (Downey, 2012)

A linguagem possui diversas estruturas e uma grande coleção de módulos prontos para uso, como listas, dicionários, data/hora, e outros recursos, inclusive é possível adicionar frameworks e bibliotecas de terceiros (Borges, 2013), além disso, python é considerado uma linguagem de multi-paradigmas, ela suporta a programação modular, procedural, funcional e a orientada à objetos (Dyer; Chauhan, 2022). Abaixo está um exemplo de um programa simples em python:

```

1 #declaração de uma variável
2 num = 5
3
4 # Verificação
5 if num > 0:
6     print("O número é positivo.")
7 elif num < 0:
8     print("O número é negativo.")

```

2.4.4.1 Pygame

Pygame é um framework para Python, foi feita sobre a biblioteca SDL(Simple DirectMedia)⁷ que permitiu abstrair os recursos da biblioteca para que fosse mais acessível e mais fácil o desenvolvimento de jogos, através de módulos é possível construir uma estrutura para um jogo (Pygame Community, 2025), todos os métodos da biblioteca são chamados pela biblioteca pygame, abaixo tem a tabela 3 exemplificando: (Pygame Community, 2025)

Tabela 3 – Funções principais da biblioteca Pygame.

Função	Descrição
<code>pygame.init()</code>	Importa e inicializa todos os módulos da biblioteca.
<code>pygame.display</code>	Vários métodos para configurar a janela ou a tela
<code>pygame.event.get()</code>	Detecta se algum evento ocorreu, por exemplo: jogador apertou uma tecla.
<code>pygame.draw</code>	Conjunto de métodos usados para desenhar formas e geometria.
<code>pygame.display.flip()</code>	Atualiza a tela inteira.
<code>pygame.quit()</code>	Fecha a aplicação.

Fonte: Elaborada pelo autor com base em (Pygame Community, 2025)

2.5 Trabalhos Correlatos

2.5.1 Desenvolvimento de jogos eletrônicos

No trabalho de (Morais, 2009) propõe um estudo de um desenvolvimento de um jogo chamado "*I Wanna be the Tribute*" feito com o motor gráficos do Game Maker 7.0, o autor aborda técnicas e o processo de engenharia de Software no desenvolvimento de um jogo.

No trabalho de (Morais, 2009) afirma que o processo de desenvolvimento de jogos eletrônicos possui 5 fases, elas são:

- **Game Design:** O *game design* é a etapa mais importante na hora de construir um jogo, com ela, é definido a forma como o jogador pensa e raciocina para tomar decisões que cheguem à um objetivo e essas decisões são tomadas a partir da regra do jogo. Segundo as palavras de (Morais, 2009):

Um design mal feito vai gerar um jogo ruim, sem dúvidas, e este problema deve-se ao fato de que o game design ainda é uma idéia

⁷ Biblioteca de desenvolvimento em C para recursos de áudio, teclado, mouse e gráficos. Disponível: <https://www.libsdl.org/>

muito nova na indústria. Um game design inovador geralmente traz um bom retorno

- **Conceitualização:** Antes de começar o desenvolvimento do jogo, o criador deve pensar como o jogo deve ser, quais os objetivos do jogador e que desafios ele irá encontrar para alcançá-los. Ao longo do desenvolvimento pode haver mudança que não está na conceitualização.
- **Imersão:** Um jogo propõe ao jogador um novo mundo fictício com regras próprias para se imergir nele, tanto o mundo quanto as regras devem fazer sentido para que seja imersivo. Um mundo estático com personagens mal planejados e sem personalidade, com uma história mal elaborada, geralmente termina em um jogo ruim.
- **Desenho:** A aparência visual de um jogo, que inclui o design dos cenários e dos personagens, é importante porque é o que permite ao jogador ver e interagir com o mundo virtual. Por isso, é essencial que a parte gráfica seja bem elaborada e bem feita.
- **Inteligência Artificial:** Nas maiorias dos jogos não há apenas o jogador naquele universo, há os não-jogadores, que são as inteligências artificiais, existem várias formas de se utilizar IA em jogos, mas o mais utilizado é o modelo básico, máquinas de estados finitos, de acordo com o ambiente e o jogador é feita uma decisão/ação.
- **Programação:** A programação é feita a partir de uma linguagem e em conjunto a linguagem é usada uma API (*Application Programming Interface*) como DirectX e OpenGL, para dar suporte à linguagem, com funções e módulos de cálculo de colisão, física de um ambiente, configuração de sons e vídeo que ajudam no desenvolvimento.

A proposta no desenvolvimento do jogo de (Morais, 2009) é que o jogo é 2D e propõe o *game design* que o jogador aprende ao perder, com a experiência da derrota anterior, o jogador obtém conhecimento do ambiente e das regras para conseguir chegar no objetivo.

A conclusão para o trabalho de (Morais, 2009) é que um projeto de desenvolver um jogo se difere em vários aspectos da engenharia de software, por necessitar da criatividade para um bom *game design*, segundo (Morais, 2009):

Um bom game design não só gera um bom jogo, mas também um bom game designer, que são aqueles que se dedicam ao estudo das técnicas propostas neste trabalho, desde a concepção das idéias até o desenho do ambiente, da interface homem máquina, e do balanceamento do jogo.

2.5.2 Estudo comparativo entre engines de desenvolvimento de jogos 2D

O trabalho de (Alves, 2020) visou comparar dois motores gráficos para o desenvolvimento de jogos 2D para celulares, Unity e a Construct 3, o jogo em si focou em ser

simples e abrangente para que as características técnicas dos motores gráficos e fossem coletadas e analisadas, tendo como objetivos específicos:

- Estudar as características técnicas das 2 *engines* para desenvolver o jogo;
- Desenvolver 2 jogos nas duas *engines*;
- Avaliar os dados analisados e coletados dos 2 jogos feitos nas 2 *engines*;

Foi feito um *Game Desing Document* para o maior entendimento do que seria o jogo à ser desenvolvido, levantando os aspectos mais importantes pro desenvolvedor. As métricas à serem avaliadas afim de comparar o desenvolvimento nas *engines* foram:

1. *TilesMap*: *Tiles* são cada bloco repetitivos que constituirão o piso e paredes de cada nível do jogo;
2. Sistema de câmera: Visão apresentado para o jogador no momento atual aonde está o personagem;
3. Movimentação do jogador: O movimento do jogador é a mudança de posição do avatar do jogador no *Tiles* recentemente criados;
4. Sistema de luz: Sistema de luz simula a luz real, em que uma fonte luz clareia uma determinada área;
5. Sistema de movimentação de IA: O movimento dos inimigos até o jogador de maneira automática;
6. Sistema de som: Processo da aplicação de tocar algum som quando determinada ação acontece;
7. Transição de Fases: Transição de uma fase do jogo para outra.

A escolhas dessas métricas segundo (Alves, 2020) está diretamente relacionado ao aspectos de desenvolvimento.

Foi desenvolvido em ambos motores gráficos um jogo chamado "A luz no fim do túnel", de gênero *puzzle*, 2D e *top-down* em que o jogador se encontrar em um labirinto com inimigos e armadilhas.

O estudo de (Alves, 2020) concluiu que o desenvolvimento na Construct 3 foi mais simples devidos algumas ferramentas presentes na *engine* e foi se utilizado scripts em JavaScript, já a Unity se mostrou mais complexa e mais completa que a Construct 3, porém devido a comunidade envolta da Unity é maior, portanto houve mais alternativas de conteúdo para resolução de problemas durante a fase de desenvolvimento. Foi se observado

também que para construção do executável do jogo, a disponibilização na Unity é gratuita, mas na Construct 3 o desenvolvedor é obrigado a pagar a licença anual.

2.5.3 Estudo comparativo de motores de jogos no desenvolvimento de um jogo 2D para web

O trabalho se inicia com (Pantoja, 2022) explicando a relevância dos jogos para web e que a partir deste cenário, o trabalho terá como objetivo desenvolver um jogo 2D avaliando os motores de jogos utilizados durante desenvolvimento, sendo assim, um estudo comparativo.

Foi feita uma pesquisa no *Google Trends* para escolher e filtrar os motores de jogos mais utilizados, quais exportam os jogos para web e quais atendem ao requisito mínimo que seria um notebook k Dell Inspiron 5537 Intel(R) Core(TM) i7-4500U CPU @ 1.80GHZ (4 CPUs) 2.40GHZ, 8GB Memória RAM, também foi feito um *Game Design Document* para levantar os requisitos e aspectos primordiais do jogo a ser desenvolvido.

Como resultado os apenas as *Unity* e *Godot* foram selecionados para o desenvolvimento do jogo. O jogo desenvolvido se chama "Os mortos-vivos" do gênero ação, no qual o jogador derrota hordas de inimigos, coleta experiência e sobe de nível.

Foram utilizados algumas métricas para realizar a avaliação dos motores, métricas para o código fonte e métricas para performance: Para o código fonte:

1. Tamanho final do arquivo;
2. Quantidade de arquivos de código;
3. Quantidade de linhas;
4. Quantidade de linhas de código;
5. Complexidade do código.

Para a performance:

1. Tempo de carregamento de jogo;
2. Uso de CPU mínimo;
3. Uso de CPU máximo;
4. Média do uso de CPU;
5. Uso de memória RAM mínimo;

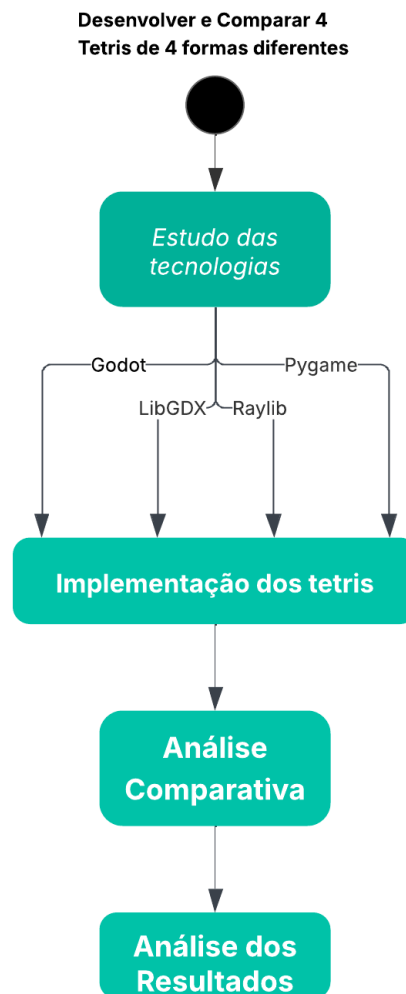
6. Uso de memória RAM máximo;
7. Média do uso de memória RAM.

O estudo comparativo de (Pantoja, 2022) foi concluído que entre os motores *Unity* e *Godot* evidencia diferenças significativas em termos de usabilidade e desempenho. O *Godot* apresenta menor complexidade, exigindo menos linhas de código e sendo mais acessível para iniciantes no desenvolvimento de jogos. Já a *Unity*, embora mais complexo, demonstrou desempenho superior durante os testes, com maior estabilidade em cenários variados. Assim, o *Godot* é recomendado para iniciantes, enquanto a *Unity* se mostra mais adequado para projetos que priorizam funcionalidades complexas e uma melhor experiência do jogador.

3 METODOLOGIA

A metodologia para alcançar os resultados do estudo comparativos, será dividido em 4 etapas, na figura 22 abaixo elas são:

Figura 22 – Diagrama da metodologia do estudo comparativo



Fonte: Feito pelo autor

3.0.1 Estudos das Tecnologias

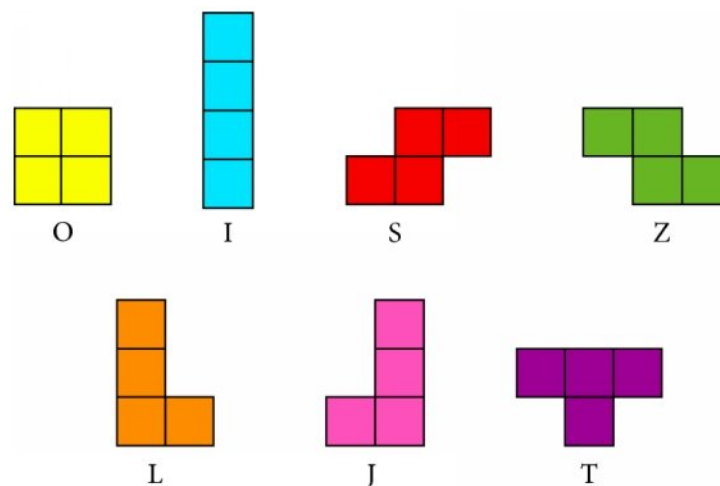
Para cada linguagem de programação e sua respectiva *engine* e seus respectivos *frameworks* (Godot, Pygame, Raylib, LibGDX) será feito um estudo inicial para aprender como funciona a estrutura de cada *framework* e seus principais métodos, o ciclo de vida de um jogo tanto da *engine* como nos *framework* é praticamente igual, com algumas diferenças

entre si, o que tornará o processo de aprendizagem menos complexo. O estudo será feito de forma prática, consultando documentações oficiais e materias complementares conforme a necessidade do desenvolvimento, a etapa do estudo das tecnologias e a implementação ocorrerá paralelamente.

3.0.2 Implementação dos *Tetris*

O *Tetris* pode ser categorizado como um jogo de quebra-cabeça, em uma grade 10 colunas por 20 linhas, o jogador empilha formas geometricas denominadas *tetraminós*, assim que uma linha inteira é preenchida, essa linha é retirada e as outras serão deslocadas para baixo, assim liberando espaço para a continuidade do jogo (Brzustowski, 1992), os *tetraminós* são formas geometricas composta sempre por 4 quadrados, há 7 *tetraminós* que o jogador pode controlar, são eles mostrado na figura 23 abaixo:

Figura 23 – Os 7 *tetraminós*



Fonte: (Lewis; Beswick, 2015)

O objetivo do jogador é evitar que a pilha de peças chegue até o topo da grade, conseguindo jogar pelo maior tempo possível (Brzustowski, 1992). A escolha do jogo *Tetris* para implementação e comparação foi por que é um jogo clássico de lógica simplificada, porém necessita recursos fundamentais que uma ferramenta de desenvolvimento de jogo oferece como por exemplo: renderização gráfica, detecção de colisão, manipulação de array de duas dimensões, controle do jogador, atualização de telas e outros recursos. Será feita a implementação do jogo 4 vezes:

- Implementação em *Godot*;
- Implementação em Java utilizando a ferramenta *LibGDX*;
- Implementação em C++ utilizando a biblioteca *Raylib*;

- Implementação em Python utilizando a biblioteca *Pygame*.

3.0.3 Análise comparativas

Após a implementação dos *Tetris* será avaliado o tempo de desenvolvimento em cada tecnologia usada, junto da avaliação da curva de aprendizado e nível de dificuldade, se a tecnologia tem recursos e funcionalidades que facilitam o desenvolvimento, também será avaliado qual o consumo de CPU em cada um dos jogos executados, consumo de memória RAM e performance em geral.

3.0.4 Resultado das análise

A partir dos dados obtidos da análise, será discutido as vantagens e as desvantagens de cada tecnologia, e qual tecnologia apresenta maior adequação para diferentes perfis de desenvolvedores, iniciantes ou avançados.

4 ANÁLISE DOS RESULTADOS

Foi desenvolvido as quatro versões do jogo *Tetris* utilizando as tecnologias *Godot*, *Pygame*, *Raylib* e *LibGDX*, após a implementação, foi feita uma análise comparativa entre as tecnologias, levando em consideração aspectos como facilidade de uso, curva de aprendizado e desempenho. Para a implementação do jogo *Tetris* foi levantado os requisitos necessários para o desenvolvimento do jogo, como a movimentação das peças, a rotação, a queda automática, o sistema de colisão e a remoção de linhas completas.

4.1 Estudos das Tecnologias e Implementação

Antes do desenvolvimento de cada versão do jogo, foi realizado um estudo introdutório da linguagem de programação utilizada pela tecnologia correspondente, buscando compreender conceitos fundamentais de sintaxe, orientação a objetos, manipulação gráfica e estrutura de execução.

Após o estudo introdutório da linguagem, foi feito um estudo da documentação oficial de cada engine ou biblioteca/*Framework*, com foco nos recursos necessários para o desenvolvimento do jogo *Tetris*, como renderização gráfica, captura de entrada do usuário, temporização e o gerenciamento de desenho de elementos.

O desenvolvimento foi realizado de forma incremental e comparativa. A cada nova implementação, aplicou-se os mesmos requisitos do jogo em diferentes ambientes de desenvolvimento. Essa abordagem possibilitou observar diferenças relacionadas à organização do código, gerenciamento do loop principal do jogo, modularização e disponibilidade de recursos oferecidos por cada tecnologia. Durante esse processo, também foram estudados e incorporados algoritmos e mecânicas adicionais presentes no *Tetris*, como o sistema de geração de peças *7-Bag* (para evitar peças iguais) e o *Super Rotation System* (SRS) (sistema de rotação para cada peça), possibilitando a evolução gradual das implementações ao longo do projeto.

Nas subseções abaixo serão mostradas como algumas funções essenciais foram implementadas de acordo com cada linguagem e tecnologia.

4.1.1 Godot

Durante o desenvolvimento do jogo na *Godot*, observou-se uma grande facilidade na criação de cenas e gerenciamento de nós, permitindo uma implementação rápida da lógica do jogo. A estrutura de nós da *Godot* facilitou a organização do código e a reutilização

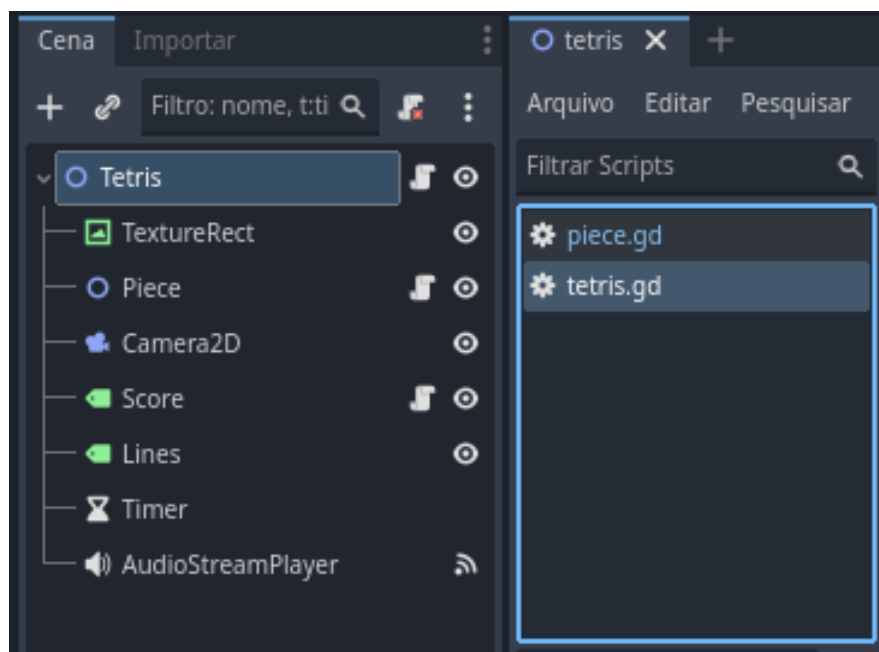
de componentes, o que contribuiu para uma implementação mais eficiente do jogo *Tetris*. A documentação da *Godot* foi útil para esclarecer dúvidas e fornecer exemplos práticos, acelerando o processo de desenvolvimento.

4.1.1.1 Nós e Scripts

Foi utilizado 2 nós principais para a implementação do jogo, o nó *Node2D* para a cena principal do jogo e outro nó *Node2D* para as peças do jogo, além de outros nós auxiliares para a temporização e gerenciamento gráfico, junto aos dois nós principais, foram criados scripts para implementar a lógica do jogo, como a movimentação das peças, a rotação, a queda automática, o sistema de colisão e a remoção de linhas completas.

Abaixo na figura 24, estão os nós e os scripts criados para a implementação do jogo *Tetris* na *Godot*:

Figura 24 – Nós e Scripts do Jogo Tetris na Godot



Fonte: Elaborado pelo autor

Os dois scripts tem dentro deles outros nós, como o nó de temporização e os nós Label Score e Lines.

4.1.1.2 Instanciação de Peças

A função de surgir as peças do jogo, chamada de *spawnPiece*, é implementada dentro do script do nó principal do jogo, o *Node2D* da cena principal, e é responsável por criar uma nova peça e posicioná-la no topo do tabuleiro.

Abaixo está implementado o código dessa função:

```
1 func spawnPiece():
```

```
2         var num = (randi() % 7 + 1)
3
4         var startPos = Vector2(GridWidth / 2 - 1, 1)
5
6         piece = Piece.new(startPos, num)
7
8         if isSpawnBlocked(piece.blocks, startPos):
9             triggerGameOver()
10            piece.queue_free
11            return
12        else:
13            add_child(piece)
14        return piece
```

Como visto no código acima, a função *spawnPiece* utiliza a função *randi* para gerar um número aleatório entre 1 e 7, que corresponde a uma das sete peças do jogo *Tetris*. Em seguida, ocorre uma condição que verifica se o espaço está disponível para a nova peça. Se sim, a função instancia a peça correspondente e a adiciona como filha do nó principal do jogo, senão, é chamada a função *gameOver*. Por fim, a função posiciona a peça no topo do tabuleiro, utilizando as coordenadas do meio do tabuleiro para centralizá-la horizontalmente e colocá-la no topo do tabuleiro. É importante observar que há possibilidade de aparecer peças iguais em sequência. Sempre quando uma peça é travada no tabuleiro, a função *spawnPiece* é chamada para criar uma nova peça.

4.1.1.3 Verificação e remoção de linhas

Quando uma linha ou mais linhas na grade é completada, a função de remoção de linhas é chamada, *cleanLines*, e é responsável por verificar quais linhas estão completas, removê-las do tabuleiro e atualizar a pontuação do jogador.

Abaixo está implementado o código dessa função:

```
1 func cleanLines():
2
3     var y = GridHeight - 1
4     var linesScores = 0
5
6     while y >= 0: # de baixo pra cima
7
8         var isFull = true
9
10        for x in range(GridWidth):
11            if grid[x][y] == 0:
12                isFull = false
13                break
14
```

```

15         if isFull:
16
17             removeLine(y)
18             linesScores += 1
19
20         else:
21             y -= 1 #checa toda a linhas de baixo pra cima
22
23     updateScore(linesScores)
24     linesScores = 0

```

A função *cleanLines* percorre cada linha do tabuleiro, verificando se todas as células da linha estão ocupadas por peças. Se uma linha completa for encontrada, é chamada a função *removeLine* para remover a linha do tabuleiro. A função *removeLine* recebe o índice da linha a ser removida. Esse processo ocorre de baixo para cima.

Abaixo está implementado o código da função *removeLine*:

```

1 func removeLine(lineY: int):
2     for y in range(lineY, 0, -1):
3         for x in range(GridWidth):
4             grid[x][y] = grid[x][y - 1]
5
6     for x in range(GridWidth):
7         grid[x][0] = 0

```

A função *removeLine* remove a linha do tabuleiro, iterando sobre as células da linha e recolocando as peças acima da linha removida para baixo.

A pontuação do jogador é atualizada com base no número de linhas removidas, seguindo a fórmula de pontuação tradicional do jogo *Tetris*, onde a pontuação aumenta exponencialmente com o número de linhas removidas simultaneamente.

4.1.1.4 Temporização e Queda Automática

A temporização na *Godot* é implementada utilizando o nó *Timer*, que é adicionado na função interna *ready*. O temporizador é configurado para disparar a cada 0.5 segundos, o que determina a velocidade de queda das peças. Abaixo está implementado o código para configurar o temporizador:

```

1 func _ready() -> void:
2
3     main_script = get_node("/root/Tetris")
4
5     var timer := $"../Timer"
6
7     timer.timeout.connect(fall)

```

```
8         timer.autostart = true
9         timer.wait_time = 0.5
10        timer.start(1)
```

A linha 7 conecta o temporizador e função responsável por fazer a peça cair *fall*, simulando a queda automática.

Abaixo está implementado o código da função *fall*:

```
1 func fall():
2
3     if not move(Vector2(0,1)):
4         checkFall += 1
5         if checkFall >= 2:
6             lockPiece(self)
7             main_script.cleanLine()
8             main_script.queue_redraw()
9             main_script.spawnPiece()
10            queue_free()
11            checkFall = 0
```

Nessa função, há uma variável chamada *checkFall* que é utilizada para verificar se a peça não pode cair, ou seja, se há colisão com o tabuleiro ou com outras peças. Se a peça não puder cair uma vez em 0.5 segundos, é adicionado 1 na variável *checkFall*. Se a variável estiver com o valor 2, é chamada a função *lockPiece* para travar a peça no tabuleiro, a função para verificar se há linhas completas e a função *spawnPiece* para criar uma nova peça e o valor da variável *checkFall* é resetado para 0.

4.1.1.5 Movimentação

A movimentação das peças é implementada utilizando a função de captura de entrada do usuário, que é chamada dentro da função *process*, responsável por atualizar o estado do jogo a cada frame. Abaixo está implementado o código para capturar a entrada do usuário:

```
1 func _process(delta: float) -> void:
2     if piece == null:
3         return
4     if Input.is_action_just_pressed("direita"):
5         piece.move(Vector2(1,0))
6     if Input.is_action_just_pressed("esquerda"):
7         piece.move(Vector2(-1,0))
8     if(Input.is_action_just_pressed("baixo")):
9         piece.move(Vector2(0,1))
10    if(Input.is_action_just_pressed("ColocarPeca")):
11        goingdown = true
12        while goingdown:
```

```

13             if piece.move(Vector2(0,1)):
14                 pass
15             else:
16                 piece.lockPiece(piece)
17                 cleanLine()
18                 queue_redraw()
19                 piece.queue_free()
20                 spawnPiece()
21                 goingdown = false
22         if(Input.is_action_just_pressed("cima")):
23             piece.rotate_piece()

```

Nessa função, são verificadas as teclas pressionadas pelo jogador para movimentar, rotacionar ou fixar a peça. A função de movimentação das peças, chamada de *move*, é implementada dentro do script do nó das peças e é responsável por atualizar a posição da peça com base na entrada do usuário. Abaixo está implementado o código da função *move*:

```

1 func move(direction: Vector2):
2     var new_position = gridPosition + direction
3     if isValidmove(new_position):
4         gridPosition = new_position
5         queue_redraw()
6         return true
7     else:
8         return false

```

Dentro da função *move*, é verificado se a nova posição da peça é válida, ou seja, se não há colisão com o tabuleiro ou com outras peças. Se a posição for válida, a peça é movida para a nova posição, caso contrário, a peça permanece na posição atual. A função que faz essa verificação é a função *isValidMove*, que é implementada dentro do script do nó das peças e é responsável por verificar se a peça colidiu com o tabuleiro ou com outras peças. Abaixo está implementado o código da função *isValidMove*:

```

1
2 func isValidmove(target_position: Vector2, target_blocks: Array[Vector2]
   = blocks) -> bool:
3     if not main_script:
4         return true
5
6     for block in target_blocks:
7         var absolute_pos = block + target_position
8         var x = int(absolute_pos.x)
9         var y = int(absolute_pos.y)
10
11         # Checar Limites Horizontais e Verticais
12         if x < 0 or x >= main_script.GridWidth:
13             return false

```

```
14         if y < 0 or y >= main_script.GridHeight:
15             return false
16
17         # Checar se a célula já está ocupada
18         if main_script.grid[x][y] != 0:
19             return false
20     return true
```

Essa função checa os limites horizontais e verticais da grade, além de verificar se a posição da peça colidiu com outras peças. Se a posição for válida, a função retorna *true*, caso contrário, retorna *false*.

A função de rotação das peças, chamada de *rotate*, também é implementada dentro do script do nó das peças, e é responsável por alterar a orientação da peça quando o jogador solicita uma rotação. Abaixo está implementado o código da função *rotate*:

```
1 func rotate_piece() -> Array[Vector2]:
2     var rotated_blocks: Array[Vector2] = []
3
4     for block in blocks:
5         var new_block: Vector2
6         new_block = Vector2(-block.y, block.x) #rotacionando
           bloco por bloco
7         rotated_blocks.append(new_block) #Coloco em um conjunto
           novo de blocos
8     if isValidmove(gridPosition, rotated_blocks):
9         blocks = rotated_blocks
10        queue_redraw()
11    return rotated_blocks
```

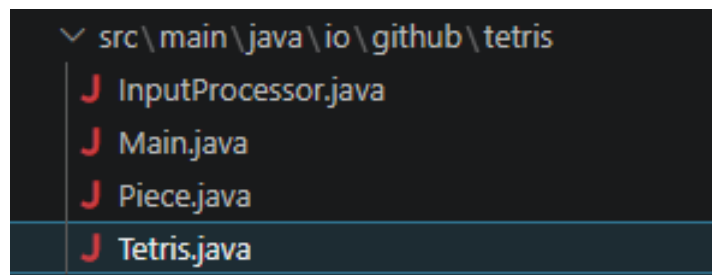
4.1.2 LibGDX

A implementação do jogo *Tetris* na *LibGDX* apresentou uma abordagem mais tradicional de desenvolvimento, utilizando a linguagem *Java*, a IDE Visual Studio Code e os recursos da engine para implementar o Tetris. Foi utilizado o paradigma de programação orientada a objetos para organizar o código, criando classes para representar as peças, o tabuleiro e a lógica do jogo. A *LibGDX* forneceu uma boa estrutura para lidar com gráficos e entrada do usuário, mas exigiu um pouco mais de configuração inicial em comparação com a *Godot*. A documentação da *LibGDX* foi útil para entender os conceitos básicos e as funcionalidades da engine, mas em alguns momentos foi necessário recorrer a fóruns e tutoriais de terceiros para esclarecer dúvidas específicas relacionadas à implementação do jogo e à recursos da engine *Tetris*.

4.1.2.1 Classes

Foi implementado 4 classes para que o jogo *Tetris* fosse implementado, a classe *Tetris* para a lógica principal do jogo e representar o tabuleiro do jogo, a classe *Piece* para representar as peças do jogo, a classe *InputProcessor* para lidar com a entrada do usuário e a classe *Main* para gerenciar o fluxo principal do jogo, na figura 25 é mostrado as classes criadas para a implementação do jogo *Tetris* na *LibGDX*:

Figura 25 – Classes do Jogo Tetris na LibGDX



Fonte: Elaborado pelo autor

Todas as classes tem dentro delas uma instância do objeto da classe *Tetris*, que é a classe principal do jogo, para que possam acessar os métodos e atributos da classe. Dentro da classe *Tetris* tem uma instância da classe *Piece* para utilizar os métodos e atributos da classe *Piece* e para o travamento das peças no tabuleiro. A classe *Main* gerencia o fluxo principal do jogo, como a renderização gráfica, a captura de entrada do usuário e a temporização.

4.1.2.2 Gerar peças

A função de gerar as peças do jogo, em comparação com a implementação na *Godot*, é implementada de uma forma diferente e mais eficiente, para evitar peças iguais. Dentro da classe *Tetris* tem a função *spawnPiece*, que é responsável por criar uma nova peça e posicioná-la no topo do tabuleiro. Abaixo está implementado o código dessa função:

```
1
2 public void spawnPiece() {
3
4     int num = getNextPieceNum();
5     Vector2 spawnPosition = new Vector2(4, 19);
6     Piece protPiece = new Piece(num, this, spawnPosition);
7     if (isSpawnBlocked(protPiece, spawnPosition)) {
8         gameOver = true;
9     } else {
10         piece = new Piece(num, this, spawnPosition);
11     }
12 }
```

A função *spawnPiece* tem uma lógica diferente da função de gerar peças da implementação na *Godot*, para evitar peças iguais, o numero para pegar a peça é gerado utilizando um sistema de *7-Bag*. O sistema de *7-Bag* é como se fosse um saco com as sete peças do jogo *Tetris*, onde cada peça é retirada do saco de forma aleatória, garantindo que todas as peças sejam geradas antes de repetir qualquer peça. Abaixo está implementado o código do sistema de *7-Bag* em 2 funções, a função *refillPieceQueue* e a função *getNextPieceNum*:

```
1 public int getNextPieceNum() {
2     System.out.println(bagPieces);
3     if (bagPieces.isEmpty()) {
4         refillPieceQueue();
5     }
6     return bagPieces.remove(0);
7 }
8
9 public void refillPieceQueue() {
10     bagPieces.add(1);
11     bagPieces.add(2);
12     bagPieces.add(3);
13     bagPieces.add(4);
14     bagPieces.add(5);
15     bagPieces.add(6);
16     bagPieces.add(7);
17     Collections.shuffle(bagPieces);
18 }
```

Há um array de inteiros chamado *bagPieces* que a função *getNextPieceNum* remove sempre o primeiro elemento da lista e retorna esse elemento como o número da peça a ser gerada. Quando a lista de peças do saco (*bagPieces*) estiver vazia, a função *refillPieceQueue* é chamada para preencher o saco com as sete peças do jogo, embaralhando com o *Collections.shuffle* a ordem das peças para garantir aleatoriedade.

4.1.2.3 Verificação e remoção de linhas

Segue a mesma lógica da implementação na *Godot*, a função de remoção de linhas é chamada sempre quando uma peça é colocada, *cleanLines*, e é responsável por verificar quais linhas estão completas, removê-las do tabuleiro e atualizar a pontuação do jogador. Abaixo está implementado o código das 2 funções, a função *cleanLines* e a função *removeLine*:

```
1 public void cleanLine() {
2
3     int linescore = 0;
4
5     for (int i = 0; i <= gridHeight - 1; i++) {
6         boolean lineFull = true;
```

```
7         for (int j = 0; j < gridWidth; j++) {
8             if (grid[j][i] == 0) {
9                 lineFull = false;
10                break;
11            }
12        }
13        if (lineFull) {
14            removeline(i--);
15            linescore++;
16            lines++;
17        }
18    }
19
20    updateScore(linescore);
21 }
22
23 public void removeline(int lineIndex) {
24     for (int i = lineIndex; i < gridHeight - 1; i++) {
25         for (int j = 0; j < gridWidth; j++) {
26             grid[j][i] = grid[j][i + 1];
27         }
28     }
29     for (int j = 0; j < gridWidth; j++) {
30         grid[j][gridHeight - 1] = 0;
31     }
32 }
```

A função *clean* percorre cada linha do tabuleiro, verificando se todas as células da linha estão ocupadas por peças. Se uma linha completa for encontrada, é chamada a função *removeLine* para remover a linha do tabuleiro. A função *removeLine* recebe o índice da linha a ser removida. Esse processo ocorre de baixo para cima. A função *removeLine* remove a linha do tabuleiro, iterando sobre as células da linha e recolocando as peças acima da linha removida para baixo.

4.1.2.4 Temporização

Abaixo está parte do código implementado para configurar a temporização do jogo, utilizando a função *render* da *LibGDX* para atualizar o estado do jogo a cada frame:

```
1 public void render() {
2     tetris.fallTimer += Gdx.graphics.getDeltaTime();
3     if (tetris.fallTimer >= tetris.fallInterval) {
4
5         tetris.fallTimer = 0;
6         tetris.lockTimer = 0;
7         tetris.piece.canFall();
8     }
```

```
9      }
10
11      if (tetris.piece.isValidMove(new Vector2(tetris.piece.position.x,
12          tetris.piece.position.y - 1),
13          tetris.piece.blocks)) {
14          tetris.lockTimer = 0;
15
16      } else {
17
18          tetris.lockTimer += Gdx.graphics.getDeltaTime();
19          if (tetris.lockTimer >= tetris.lockDelay) {
20              tetris.lockPiece();
21              tetris.spawnPiece();
22              tetris.lockTimer = 0;
23
24          }
25      }
26 }
```

Nessa função, a variável *fallTimer* é incrementada com o tempo decorrido desde o último frame, utilizando o método *Gdx.graphics.getDeltaTime* para obter o tempo em segundos. Quando o *fallTimer* atinge ou ultrapassa o valor definido em *fallInterval* (0.5 segundos), o temporizador é resetado para zero e a função responsável por fazer a peça cair, chamada de *canfall*, é chamada para simular a queda automática das peças. Junto também tem a variável *lockTimer* que é utilizada para verificar se a peça não pode cair, ou seja, se há colisão com o tabuleiro ou com outras peças. Se a peça não puder cair uma vez em 0.5 segundos, é adicionado 1 na variável *lockTimer*. Se a variável estiver com o valor maior que o limite *lockDelay*, é chamada a função *lockPiece* para travar a peça no tabuleiro.

4.1.2.5 Movimentação

A movimentação das peças é implementada utilizando a classe *InputProcessor* para lidar com a entrada do usuário, que é chamada dentro da função *render*, responsável por atualizar o estado do jogo a cada frame. Abaixo está implementado o código para capturar a entrada do usuário:

```
1 public class InputProcessor extends InputAdapter {
2     private Tetris tetris;
3
4     public InputProcessor(Tetris tetris) {
5         this.tetris = tetris;
6     }
7 }
```

```

8      @Override
9      public boolean keyDown(int keycode) {
10
11          if (keycode == Input.Keys.UP || keycode == Input.Keys.W) {
12              tetris.piece.blocks = tetris.piece.rotate();
13          }
14
15          if (keycode == Input.Keys.DOWN || keycode == Input.Keys.S) {
16              tetris.piece.move(new Vector2(0, -1));
17          }
18
19          if (keycode == Input.Keys.LEFT || keycode == Input.Keys.A) {
20              tetris.piece.move(new Vector2(-1, 0));
21          }
22
23          if (keycode == Input.Keys.RIGHT || keycode == Input.Keys.D) {
24              tetris.piece.move(new Vector2(1, 0));
25          }
26
27          if (keycode == Input.Keys.SPACE) {
28              tetris.hardDrop();
29              tetris.lockPiece();
30              tetris.spawnPiece();
31          }
32          return false;
33      }
34
35  }
```

Nessa função, são verificadas as teclas pressionadas pelo jogador para movimentar, rotacionar ou fixar a peça. A função de movimentação das peças, chamada de *move*, é implementada dentro da classe *Piece* e é responsável por atualizar a posição da peça com base na entrada do usuário. Abaixo está implementado o código da função *move*:

```

1  public void move(Vector2 direction) {
2      Vector2 newPosition = new Vector2(position.x + direction.x, position
        .y + direction.y);
3      if (!isValidMove(newPosition, blocks)) {
4          return;
5      }
6      position.x += direction.x;
7      position.y += direction.y;
8      tetris.lockTimer = 0;
9  }
```

Dentro da função *move*, é verificado se a nova posição da peça é válida, ou seja, se não há colisão com o tabuleiro ou com outras peças. Se a posição for válida, a peça é

movida para a nova posição, caso contrário, a peça permanece na posição atual. A função que faz essa verificação é a função *isValidMove*, que é implementada dentro da classe *Piece* e é responsável por verificar se a peça colidiu com o tabuleiro ou com outras peças. Abaixo está implementado o código da função *isValidMove*:

```
1 public boolean isValidMove(Vector2 targetPosition, Vector2[]
    targetBlocks) {
2     for (Vector2 block : targetBlocks) {
3         int newX = (int) (targetPosition.x + block.x);
4         int newY = (int) (targetPosition.y + block.y);
5
6         if (newX < 0 || newX >= tetris.gridWidth || newY < 0) {
7             return false;
8         }
9         if (newY >= tetris.gridHeight) {
10             continue;
11         }
12
13         if (tetris.grid[newX][newY] > 0) {
14             return false;
15         }
16     }
17     return true;
18 }
```

Essa função checa os limites horizontais e verticais da grade, além de verificar se a posição da peça colidiu com outras peças. Se a posição for válida, a função retorna *true*, caso contrário, retorna *false*. A função de rotação das peças, chamada de *rotate*, também é implementada dentro da classe *Piece*, e é responsável por alterar a orientação da peça quando o jogador solicita uma rotação. Abaixo está implementado o código da função *rotate*:

```
1 public Vector2[] rotate() {
2     Vector2[] newBlocks = new Vector2[blocks.length];
3     for (int i = 0; i < blocks.length; i++) {
4         newBlocks[i] = new Vector2(blocks[i].y, -blocks[i].x);
5     }
6     if (isValidMove(position, newBlocks)) {
7         tetris.lockTimer = 0;
8         return newBlocks;
9     }
10    if (isValidMove(new Vector2(position.x + 1, position.y), newBlocks))
11        {
12        position.x += 1;
13        tetris.lockTimer = 0;
14        return newBlocks;
15    }
```

```
15     if (isValidMove(new Vector2(position.x - 1, position.y), newBlocks))
16     {
17         position.x -= 1;
18         tetris.lockTimer = 0;
19         return newBlocks;
20     }
21     if (isValidMove(new Vector2(position.x, position.y + 1), newBlocks))
22     {
23         position.y += 1;
24         tetris.lockTimer = 0;
25         return newBlocks;
26     }
27     return blocks;
```

O método de rotação utilizado é uma versão simplificada do *Super Rotation System* (SRS), que é um sistema de rotação para cada peça, onde cada peça tem um conjunto específico de regras para rotação, permitindo uma rotação mais fluida e intuitiva, especialmente em situações de colisão. Caso a peça não possa ser rotacionada devido a uma colisão, o sistema de rotação tenta aplicar um deslocamento (chamado de "kick") para tentar encontrar uma posição válida para a peça rotacionada, é feito 4 tentativas. Se nenhuma posição válida for encontrada, a peça permanece na orientação atual.

4.1.3 Raylib

A implementação do jogo *Tetris* na *Raylib* apresentou uma abordagem mais direta e de baixo nível, foi utilizado também o Visual Studio Code como ambiente de desenvolvimento, usou-se a linguagem *C++* e os recursos do *Framework* para implementar o Tetris. A *Raylib* forneceu uma boa estrutura para lidar com gráficos e entrada do usuário, mas exigiu um pouco mais de configuração inicial em comparação com a *Godot* e a *LibGDX*. A documentação da *Raylib* foi útil para entender os conceitos básicos e as funcionalidades do *Framework*, porém, em alguns momentos foi necessário recorrer a fóruns e tutoriais de terceiros para esclarecer dúvidas específicas relacionadas à linguagem *C++*, como por exemplo, o uso de ponteiros e referências.

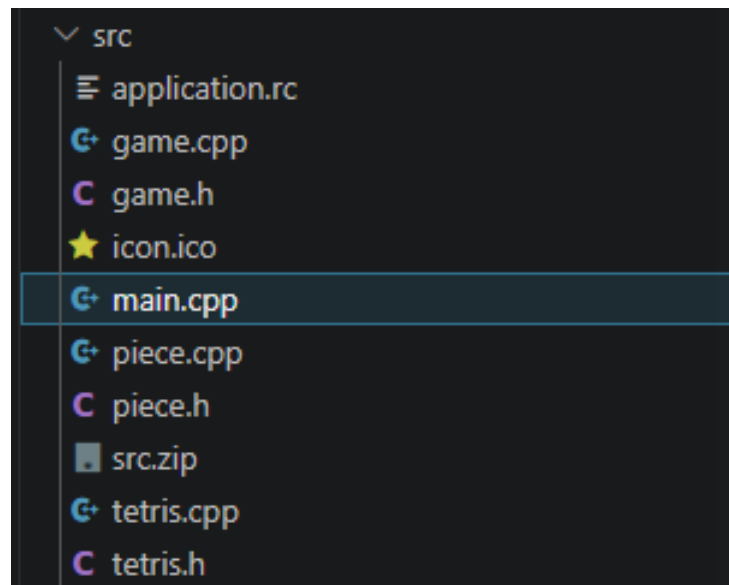
Há também na documentação oficial exemplos prontos de implementação de jogos, incluindo uma versão funcional do jogo *Tetris*. Entretanto, esse exemplo não foi utilizado diretamente no desenvolvimento do projeto, por que o objetivo do trabalho consistia em compreender e implementar manualmente as mecânicas do jogo, permitindo maior aprofundamento nos conceitos da linguagem *C++* e da própria biblioteca.

4.1.3.1 Classes

Foi implementado 4 classes para essa versão, a classe *Tetris* para a lógica principal do jogo e representar o tabuleiro do jogo, a classe *Piece* para representar as peças do jogo, a classe *Game* para gerenciar o fluxo principal do jogo, como a renderização gráfica, a captura de entrada do usuário e a temporização, e a classe principal presente em *main.cpp*, utilizada para inicialização da implementação. Além disso, a implementação utilizou arquivos de cabeçalho (*headers*), como *piece.h*, *tetris.h* e *game.h*, responsáveis pela declaração das classes, atributos e métodos utilizados no projeto. Já os arquivos com extensão *.cpp* foram utilizados para a implementação da lógica dessas classes. Essa separação entre declaração e implementação contribuiu para uma melhor organização e modularização do código, característica comum em projetos desenvolvidos em *C++*.

Como *C++* requer um gerenciamento de memória manual, foi necessário lidar com a gestão de memória manualmente, utilizando ponteiros e referências para criar e destruir objetos dinamicamente. Na figura 26 é mostrado as classes criadas para a implementação do jogo *Tetris* na *Raylib*:

Figura 26 – Classes e Headers do Jogo Tetris na Raylib



Fonte: Elaborado pelo autor

4.1.3.2 Gerar Peça

O código para gerar as peças do jogo na *Raylib* segue a mesma lógica da implementação da *LibGDX*, utilizando o algoritmo *7-bag*, que é implementado dentro da classe *Tetris* e é responsável por criar uma nova peça e posicioná-la no topo do tabuleiro. Abaixo está implementado o código dessas funções:

```
1 void Tetris::spawnPiece() {  
2     if (piece != nullptr) {
```

```
3         delete piece;
4         piece = nullptr;
5     }
6
7     Vector2 spawnPosition = {4,0};
8     if(bagPieces.empty()){
9         refillBag();
10    }
11
12    int num = bagPieces.back();
13    bagPieces.pop_back();
14
15    Piece newPiece(this, num, spawnPosition);
16    if(isSpawnBlocked(&newPiece, spawnPosition)){
17        gameover = true;
18        piece = nullptr;
19    } else {
20        piece = new Piece(this, num, spawnPosition);
21    }
22 }
23
24 void Tetris::refillBag(){
25     bagPieces.push_back(1);
26     bagPieces.push_back(2);
27     bagPieces.push_back(3);
28     bagPieces.push_back(4);
29     bagPieces.push_back(5);
30     bagPieces.push_back(6);
31     bagPieces.push_back(7);
32     shuffleBag();
33 }
34
35 void Tetris::shuffleBag(){
36     std::shuffle(bagPieces.begin(), bagPieces.end(), generator);
37 }
```

Tem um array de inteiros chamado *bagPieces*, é removido o último elemento da lista e retorna esse elemento como o número da peça a ser gerada. Quando a *bagPieces* estiver vazia, a função *refillBag* é chamada para preencher o saco com as sete peças do jogo, embaralhando com a função *std::shuffle* a ordem das peças para garantir aleatoriedade. Logo apos é verificado se o espaço está disponível para a nova peça. Se sim, a função instancia a peça correspondente, senão, o jogo é encerrado. É importante observar que foi utilizado ponteiros na hora de verificar o espaço, para que a verificação seja feita por referência e não por valor.

4.1.3.3 Verificação e remoção de linhas

A função de remoção de linhas também segue o mesmo princípio da implementação da *LibGDX*. Ela é chamada sempre quando uma peça é colocada, *cleanLines*, e é responsável por verificar quais linhas estão completas, removê-las do tabuleiro e atualizar a pontuação do jogador. Abaixo está implementado o código das 2 funções, a função *cleanLines* e a função *removeLine*:

```
1 void Tetris::cleanLine(){
2     int linesCleared = 0;
3     for(int i = 0; i < height; i++){
4         bool fullLine = true;
5         for(int j = 0; j < width; j++){
6             if(grid[i][j] == 0){
7                 fullLine = false;
8                 break;
9             }
10        }
11        if(fullLine){
12            removeLine(i);
13            linesCleared++;
14            lines++;
15        }
16    }
17    addScore(linesCleared);
18 }
19
20 void Tetris::removeLine(int line){
21     for(int i = line; i > 0; i--){
22         for(int j = 0; j < width; j++){
23             grid[i][j] = grid[i-1][j];
24         }
25     }
26     for(int j = 0; j < width; j++){
27         grid[0][j] = 0;
28     }
29 }
```

A função *cleanLine* percorre cada linha do tabuleiro, verificando se todas as células da linha estão ocupadas por peças. Se uma linha completa for encontrada, é chamada a função *removeLine* para remover a linha do tabuleiro. A função *removeLine* recebe o índice da linha a ser removida. Esse processo ocorre de baixo para cima. A função *removeLine* remove a linha do tabuleiro, iterando sobre as células da linha e recolocando as peças acima da linha removida para baixo.

4.1.3.4 Temporização e Queda Automática

Na implementação utilizando a *Raylib*, a temporização foi realizada através da atualização contínua do estado do jogo dentro do *game loop*. A cada iteração do laço principal, a função *update* da classe *Game* é chamada para atualizar a lógica do jogo.

Abaixo está implementado o código responsável pela atualização do estado do jogo:

```
1 void Game::update(){
2     if(!tetris.gameover){
3         tetris.gravity();
4         tetris.checkLock();
5     } else{
6         tetris.gameOver();
7     }
8 }
```

Nessa função, é verificado se o *Game Over* não está ativo quando o jogo está em execução, são chamadas as funções *gravity* e *checkLock*, responsáveis pela queda automática das peças e pela verificação do travamento no tabuleiro.

A função *gravity* é responsável por controlar a velocidade de queda das peças através da variável *fallTimer*:

```
1 void Tetris::gravity(){
2     if(gameover || piece == nullptr) return;
3
4     fallTimer++;
5     if(fallTimer >= fallDelay){
6         fallTimer = 0;
7         lockTimer = 0;
8         piece->canFall();
9     }
10 }
```

A cada atualização do jogo, a variável *fallTimer* é incrementada. Quando seu valor atinge o limite definido em *fallDelay*, o temporizador é reiniciado e a função *canFall* da peça atual é chamada, simulando sua queda automática no tabuleiro.

Além disso, a função *checkLock* é utilizada para verificar se a peça atual ainda pode se mover para baixo:

```
1 void Tetris::checkLock(){
2     if(gameover || piece == nullptr) return;
3
4     if(!piece->isValidMove({0,1}, piece->blocks)){
5         lockTimer++;
6         if(lockTimer >= lockDelay){
7             lockTimer = 0;
```

```

8         lockPiece();
9         spawnPiece();
10    }
11 }
12 }

```

Caso a função *isValidMove* indique que não há espaço livre abaixo da peça, a variável *lockTimer* é incrementada. Quando esse valor atinge o limite definido em *lockDelay* (0.5 segundos), a função *lockPiece* é chamada para fixar a peça no tabuleiro, seguida pela criação de uma nova peça através da função *spawnPiece*.

Diferentemente da implementação em *Godot*, que utiliza um nó *Timer* para controlar a queda das peças, e da implementação em *LibGDX*, que utiliza o tempo decorrido entre quadros (*deltaTime*), a implementação em *Raylib* utiliza a própria atualização contínua no loop principal do jogo para realizar o controle da temporização e da queda automática das peças.

4.1.3.5 Movimentação

A movimentação das peças na implementação utilizando a *Raylib* é realizada através da função *inputs*, presente na classe *Game*. Essa função é chamada a cada iteração do *game loop*, sendo responsável por capturar as entradas do usuário e executar as ações correspondentes.

Abaixo está implementado o código da função *inputs*:

```

1 void Game::inputs(){
2
3     if(tetris.piece == nullptr) return;
4     if(IsKeyPressed(KEY_LEFT) || IsKeyPressed(KEY_A)){
5         tetris.piece->move({-1, 0});
6         tetris.lockTimer = 0;
7     }
8     if(IsKeyPressed(KEY_RIGHT) || IsKeyPressed(KEY_D)){
9         tetris.piece->move({1, 0});
10        tetris.lockTimer = 0;
11    }
12    if(IsKeyPressed(KEY_DOWN) || IsKeyPressed(KEY_S)){
13        tetris.piece->move({0, 1});
14        tetris.lockTimer = 0;
15    }
16    if(IsKeyPressed(KEY_UP) || IsKeyPressed(KEY_W)){
17        tetris.piece->rotate();
18        tetris.lockTimer = 0;
19    }
20    if(IsKeyPressed(KEY_SPACE)){
21        tetris.piece->hardDrop();

```

```
22         tetris.lockPiece();
23         tetris.piece->~Piece();
24         tetris.spawnPiece();
25     }
26 }
```

Nessa função, são verificadas as teclas pressionadas pelo jogador para movimentar, rotacionar ou realizar a queda instantânea (*Hard Drop*) da peça atual. Quando uma tecla de movimentação é pressionada, é chamada a função *move* da classe *Piece*, responsável por atualizar a posição da peça no tabuleiro. Além disso, o temporizador de travamento (*lockTimer*) é reiniciado, permitindo que o jogador continue ajustando a posição da peça após o contato com outras peças ou com o fundo do tabuleiro.

Observa-se que é realizada a destruição da instância atual da peça através da chamada ao destrutor da classe *Piece*, após a peça ser posicionada instantaneamente no tabuleiro e travada através da função *lockPiece*, para garantir que a memória alocada para a peça seja liberada corretamente, evitando vazamentos de memória. Essa abordagem é necessária devido à natureza de gerenciamento manual de memória em *C++*, onde o desenvolvedor é responsável por alocar e liberar memória de forma explícita.

A função responsável pela movimentação das peças é apresentada abaixo:

```
1 void Piece::move(Vector2 direction){
2     if(pointerToTetris == nullptr){
3         return;
4     }
5     Vector2 newPosition = {position.x + direction.x, position.y +
6                             direction.y};
7     if(isValidMove(direction, blocks)){
8         position = newPosition;
9     }
10 }
```

Dentro da função *move*, é calculada uma nova posição para a peça a partir da direção informada pelo jogador. Antes de atualizar a posição da peça, é realizada uma verificação através da função *isValidMove*, que determina se o movimento é válido. Caso não exista colisão com os limites do tabuleiro ou com outras peças já posicionadas, a nova posição é aplicada, caso o contrário, a peça permanece na posição atual.

A função *isValidMove* é responsável por verificar se a movimentação ou rotação da peça é permitida, verificando os limites horizontais e verticais do tabuleiro e possíveis colisões com peças já travadas.

A rotação também é implementada na classe *Piece*, através da função *rotate*, apresentada abaixo:

```
1 void Piece::rotate(){
2     if(num == 4){
3         return;
4     }
5     Vector2 newBlocks[4];
6     for(int i = 0; i < 4; i++){
7         newBlocks[i].x = -blocks[i].y;
8         newBlocks[i].y = blocks[i].x;
9     }
10    if(isValidMove({0,0}, newBlocks)){
11        for(int i = 0; i < 4; i++){
12            blocks[i] = newBlocks[i];
13        }
14        return ;
15    }
16    if(isValidMove({1,0}, newBlocks)){
17        for(int i = 0; i < 4; i++){
18            blocks[i] = newBlocks[i];
19        }
20        position.x += 1;
21        return ;
22    }
23    if(isValidMove({-1,0}, newBlocks)){
24        for(int i = 0; i < 4; i++){
25            blocks[i] = newBlocks[i];
26        }
27        position.x -= 1;
28        return;
29    }
30    if(isValidMove({0,1}, newBlocks)){
31        for(int i = 0; i < 4; i++){
32            blocks[i] = newBlocks[i];
33        }
34        position.y += 1;
35        return;
36    }
37 }
```

O algoritmo de rotação utilizado é uma versão simplificada do *Super Rotation System* (SRS). Inicialmente, a rotação é realizada utilizando uma transformação geométrica que converte cada bloco da peça para sua nova posição rotacionada. Após a rotação, é verificado se a nova orientação é válida na posição atual.

Caso a rotação resulte em colisão, o sistema realiza tentativas de deslocamento da peça para encontrar uma posição válida. Se nenhuma posição válida for encontrada, a peça permanece em sua posição original.

4.1.4 Pygame

A implementação do jogo *Tetris* na *Pygame* foi a mais simples e direta, também utilizando o Visual Studio Code como ambiente de desenvolvimento, usou-se a linguagem *Python* e os recursos da biblioteca para implementar o Tetris. A *Pygame* forneceu uma boa estrutura para lidar com gráficos e entrada do usuário, e exigiu uma configuração inicial mínima em comparação com as outras tecnologias. A documentação da *Pygame* foi útil para entender os conceitos básicos e as funcionalidades da biblioteca. Na implementação, foi utilizado o paradigma de programação procedural para organizar o código, criando funções para representar as peças, o tabuleiro e a lógica do jogo. A estrutura do código foi mais linear e menos modularizada em comparação com as outras tecnologias, mas ainda assim permitiu uma implementação funcional do jogo *Tetris*. Além disso foi utilizado o Venv para criar um ambiente virtual e gerenciar as dependências do projeto, como a instalação da biblioteca *Pygame*.

4.1.4.1 Gerar peças

A geração de peça segue a mesma lógica da implementação na *LibGDX* e na *Raylib*, utiliza-se também o algoritmo *7-bag*. A função de gerar as peças do jogo, chamada de *spawnPiece*, é responsável por criar uma nova peça e posicioná-la no topo do tabuleiro.

Abaixo está implementado o código do *7-Bag*:

```
1 def spawnPiece(self):
2     nextPiece = Piece(self.getNextPiece(), Vector2(5,1))
3     if self.isSpawnBlocked(nextPiece):
4         self.gameOver()
5         return
6     else:
7         self.piece = nextPiece
8
9 def getNextPiece(self):
10     if len(self.bag) == 0:
11         self.refillBag()
12     return self.bag.pop()
13
14 def refillBag(self):
15     self.bag = [1,2,3,4,5,6,7]
16     random.shuffle(self.bag)
```

A peça é gerada utilizando o sistema de *7-Bag*, onde um número aleatório é retirado de uma lista que contém as sete peças do jogo. Quando o array de nome *bag* estiver vazio, a função *refillBag* é chamada para preencher o saco com as sete peças do jogo, embaralhando com a função *random.shuffle* a ordem das peças para garantir aleatoriedade. Logo após

isso, é verificado se o espaço está disponível para a nova peça. Se sim, a função instancia a peça correspondente, senão, o jogo é encerrado.

4.1.4.2 Verificação e Remoção de linha

A função de remoção de linhas segue a mesma lógica da implementação na *Godot*, *LibGDX* e *Raylib*. Ela é chamada sempre quando uma peça é colocada, *clearLines*, e é responsável por verificar quais linhas estão completas, removê-las do tabuleiro e atualizar a pontuação do jogador.

Abaixo está implementado o código dessa função:

```
1 def clearLines(self):
2     linesCleared = 0
3     for y in range(len(self.grid)):
4         if 0 not in self.grid[y]:
5             del self.grid[y]
6             self.grid.insert(0, [0 for x in range(self.width)])
7             linesCleared += 1
8
9     if linesCleared > 0:
10         self.updateScore(linesCleared)
```

A função percorre todas as linhas do tabuleiro verificando se existem espaços vazios, representados pelo valor *0*. Quando uma linha não contém nenhum espaço vazio, ela é considerada completa e é removida do tabuleiro através do comando *del*. Em seguida, uma nova linha vazia é inserida na parte superior da grade utilizando o método *insert*, simulando o deslocamento das demais linhas para baixo.

Durante esse processo, a variável *linesCleared* é utilizada para contabilizar a quantidade de linhas removidas em uma única jogada. Após a verificação de todas as linhas do tabuleiro, caso pelo menos uma linha tenha sido removida, a função *updateScore* é chamada para atualizar a pontuação do jogador de acordo com a quantidade de linhas eliminadas.

4.1.4.3 Temporização e Queda Automática

A temporização na implementação utilizando a *Pygame* é chamada dentro do loop principal do jogo para controlar a taxa de atualização dos frames, a abordagem é similar à implementação da *LibGDX*.

Abaixo está implementado o código para a temporização:

```
1 while running:
2     for event in pygame.event.get():
3         if event.type == pygame.QUIT:
4             running = False
```

```
5
6     if not gameOverRunning:
7         fallTimer += dt
8         if fallTimer >= fallInterval:
9             #print("1 second")
10            fallTimer = 0
11            if grid.piece.isValidMove(Vector2(0,1)):
12                grid.piece.move(Vector2(0,1))
13
14            if grid.piece.isValidMove(Vector2(0,1)):
15                lockTimer = 0
16            else:
17                lockTimer += dt
18                if lockTimer >= lockInterval:
19                    grid.lockPiece()
20                    grid.spawnPiece()
21                    lockTimer = 0
22
23            dt = clock.tick(60) / 1000
24
25 pygame.quit()
```

A variável *dt* é obtida através da função *clock.tick(60)*, responsável por limitar a execução do jogo a 60 quadros por segundo. O valor retornado é convertido para segundos e utilizado para controlar a temporização das mecânicas do jogo de forma independente da taxa de quadros.

Para implementar a queda automática das peças, foi utilizada a variável *fallTimer*, que acumula o tempo decorrido a cada iteração do laço principal. Quando esse valor atinge o intervalo definido em *fallInterval*, o temporizador é reiniciado e é verificado se a peça atual pode se mover para baixo através da função *isValidMove*. Caso o movimento seja válido, a função *move* é chamada para deslocar a peça uma posição para baixo no tabuleiro.

Além da queda automática, a implementação também utiliza a variável *lockTimer*, responsável por controlar o travamento das peças. Enquanto a peça puder continuar descendo, o valor de *lockTimer* permanece zerado. Quando a peça não possui mais espaço livre abaixo dela, o temporizador começa a ser incrementado utilizando o valor de *dt*.

Quando o valor de *lockTimer* atinge o limite definido em *lockInterval*, a função *lockPiece* é chamada para fixar a peça no tabuleiro e, em seguida, a função *spawnPiece* cria uma nova peça para dar continuidade à partida. Após isso, o temporizador é reiniciado.

4.1.4.4 Movimentação

A movimentação das peças na *Pygame* é semelhante a *Godot*, *LibGDX* e na *Raylib*, implementada na classe responsável por representar as peças do jogo, a *Piece*.

A função *move* recebe como parâmetro um vetor indicando a direção do movimento desejado. Abaixo está implementado o código da função *move*:

```
1 def move(self, direction):
2     if self.isValidMove(direction):
3         self.position.x += direction.x
4         self.position.y += direction.y
```

Antes de atualizar a posição da peça, a função verifica se o movimento solicitado é válido através da função *isValidMove*. Caso não exista colisão com os limites do tabuleiro ou com outras peças já posicionadas, as coordenadas da peça são atualizadas de acordo com a direção informada.

A validação dos movimentos é realizada pela função *isValidMove*, apresentada abaixo:

```
1 def isValidMove(self, direction, blocks=None):
2
3     if blocks is None:
4         blocks = self.blocks
5
6     for block in blocks:
7         gridX = int(self.position.x + block.x + direction.x)
8         gridY = int(self.position.y + block.y + direction.y)
9
10        if gridX < 0 or gridX >= width or gridY >= height:
11            return False
12
13        if grid.grid[gridY][gridX] != 0:
14            return False
15
16    return True
```

Essa função percorre todos os blocos que compõem a peça atual, calculando suas posições na grade após a aplicação do movimento solicitado. Em seguida, são verificadas possíveis colisões com os limites horizontais e verticais do tabuleiro, bem como colisões com blocos já existentes na matriz do jogo. Caso alguma dessas condições seja encontrada, a função retorna *False*; caso contrário, retorna *True*, indicando que o movimento pode ser realizado.

A rotação das peças também é implementada na própria classe da peça através da função *rotate*, apresentada abaixo:

```
1 def rotate(self):
2     if self.num == 7:
3         return
4     newBlocks = []
```

```
5     for block in self.blocks:
6         newX = -block.y
7         newY = block.x
8         newBlocks.append(Vector2(newX, newY))
9
10    if self.isValidMove(Vector2(0,0), newBlocks):
11        self.blocks = newBlocks
12        return newBlocks
13    elif self.isValidMove(Vector2(1,0), newBlocks):
14        self.position.x += 1
15        self.blocks = newBlocks
16        return newBlocks
17    elif self.isValidMove(Vector2(-1,0), newBlocks):
18        self.position.x -= 1
19        self.blocks = newBlocks
20        return newBlocks
21    elif self.isValidMove(Vector2(0,1), newBlocks):
22        self.position.y += 1
23        self.blocks = newBlocks
24        return newBlocks
25    return self.blocks
```

O método de rotação utilizado consiste em aplicar uma transformação geométrica sobre cada bloco da peça, calculando sua nova posição após uma rotação de 90 graus. Após a geração da nova configuração dos blocos, o sistema verifica se a peça rotacionada pode ocupar a posição atual do tabuleiro.

Caso a rotação resulte em colisão, são realizadas tentativas de deslocamento da peça para posições próximas, verificando a posição original, uma posição à direita, uma posição à esquerda e uma posição abaixo da posição atual. Caso alguma dessas alternativas seja válida, a rotação é aplicada. Caso contrário, a peça permanece em sua posição original.

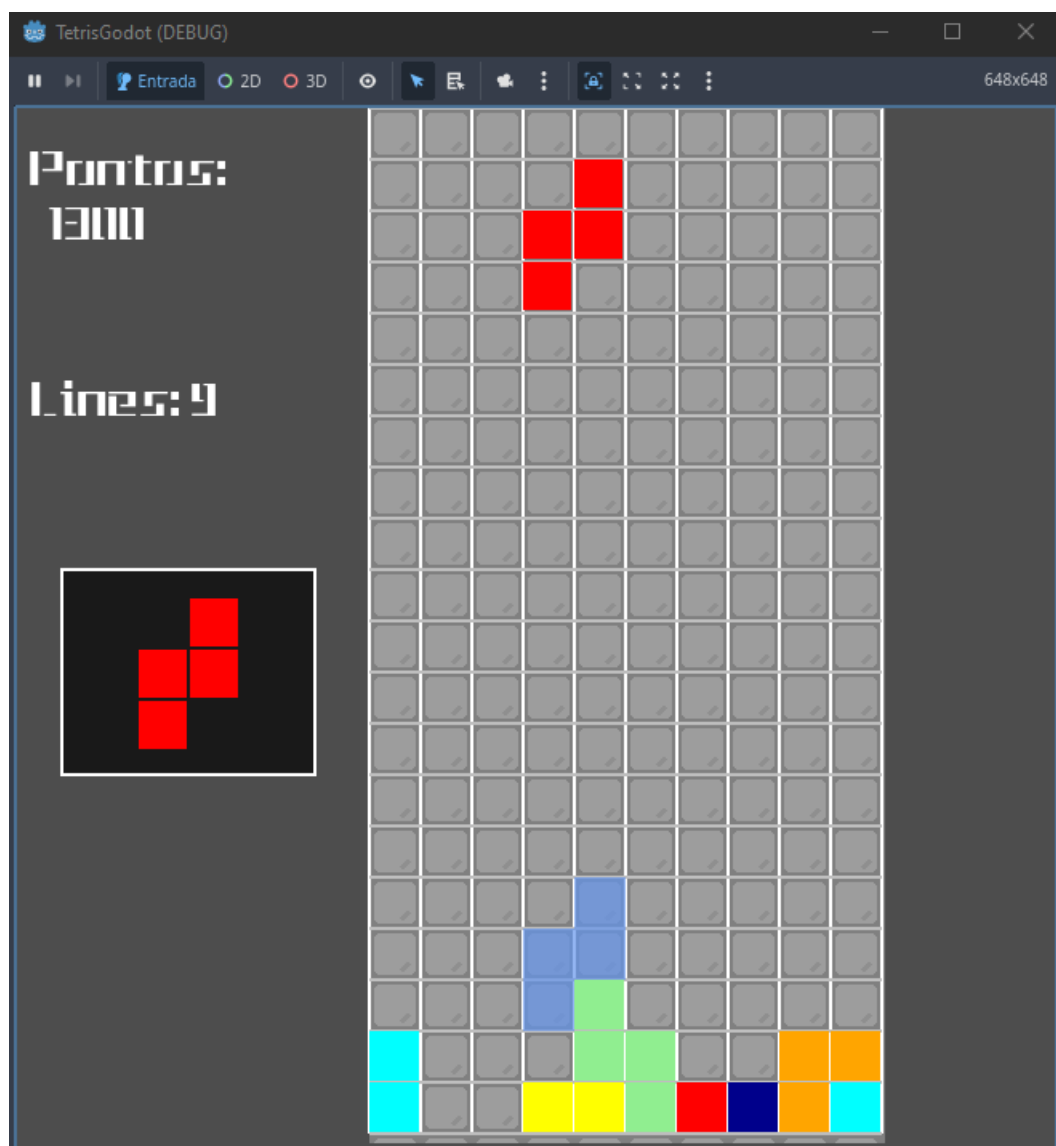
4.2 Resultados/Impactos

Apesar da evolução gradual a cada implementação do jogo *Tetris*, foi unificado o comportamento das 4 versões, garantindo que a experiência do usuário seja idêntica em todas as versões (mesma velocidade, cores semelhantes, mesma lógica de pontuação).

Também foi possível observar que cada uma delas apresentou características distintas em termos de estrutura de desenvolvimento, modularização do código, facilidade de implementação e nível de abstração oferecido.

Após as implementações do jogo *Tetris* utilizando as quatro tecnologias selecionadas, foi possível observar diferenças mínimas na questão gráfica. Nas figuras 27, 28, 29 e 30 abaixo como o jogo ficou visualmente em cada tecnologia:

Figura 27 – Jogo Tetris na Godot



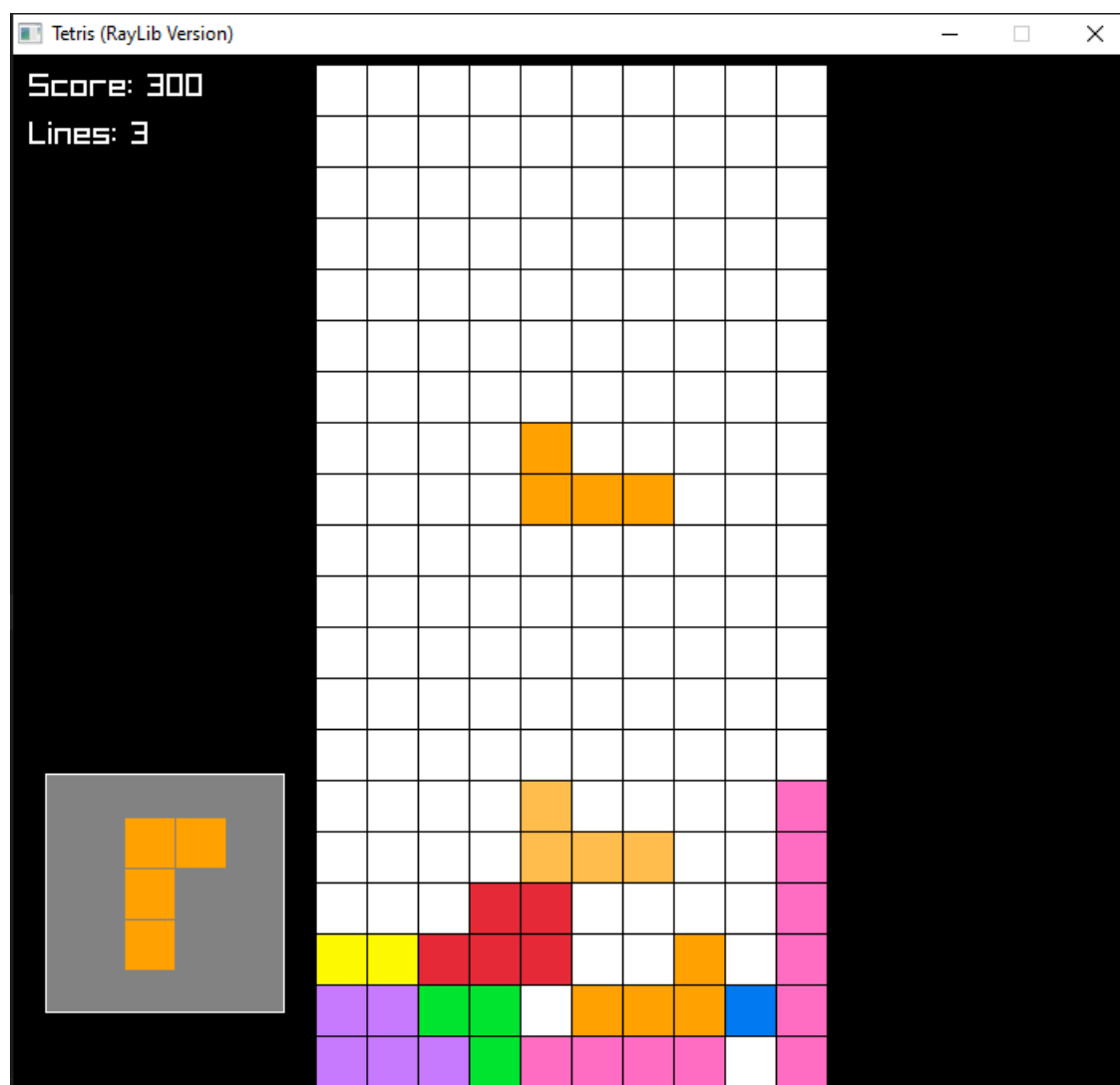
Fonte: Elaborado pelo autor

Figura 28 – Jogo Tetris na LibGDX



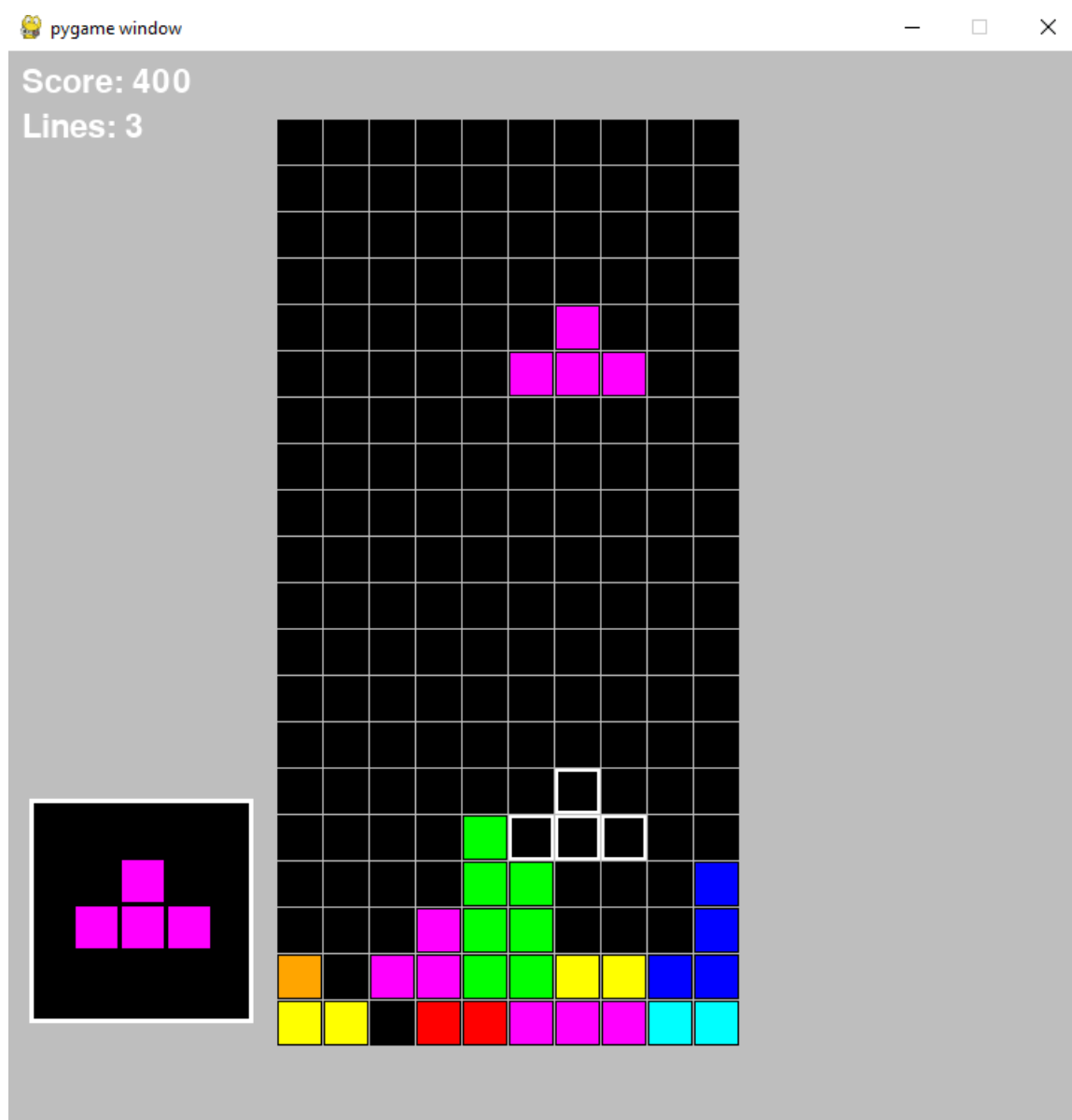
Fonte: Elaborado pelo autor

Figura 29 – Jogo Tetris na Raylib



Fonte: Elaborado pelo autor

Figura 30 – Tetris na Pygame



Fonte: Elaborado pelo autor

Todos os códigos-fonte desenvolvidos estão disponibilizados em repositórios públicos na plataforma GitHub. Abaixo estão os links para os repositórios de cada implementação do jogo *Tetris* utilizando as tecnologias selecionadas:

- **Tetris Godot:** <<https://github.com/Samuskox/Tetris-Godot>>
- **Tetris LibGDX (Java):** <<https://github.com/Samuskox/Tetris-LibGDX>>
- **Tetris Raylib (C++):** <<https://github.com/Samuskox/Tetris-Raylib>>
- **Tetris Pygame (Python):** <<https://github.com/Samuskox/Tetris-Pygame>>

4.2.1 Análise Comparativa

A análise comparativa entre as tecnologias utilizadas permitiu observar diferenças significativas relacionadas à estrutura de desenvolvimento, modularização do código, facilidade de implementação e nível de abstração oferecido por cada ferramenta. Apesar de todas as tecnologias terem sido capazes de implementar os requisitos funcionais do jogo *Tetris*, cada uma apresentou vantagens e limitações específicas durante o processo de desenvolvimento.

4.2.1.1 Facilidade de Desenvolvimento

A *Godot* e a *Pygame* apresentaram menor complexidade inicial de desenvolvimento, permitindo uma implementação mais rápida das mecânicas básicas do jogo. A estrutura baseada em nós da *Godot* simplificou o gerenciamento dos elementos gráficos e temporizadores, enquanto a flexibilidade da linguagem *Python* tornou o desenvolvimento na *Pygame* mais acessível.

Por outro lado, a *LibGDX* e a *Raylib* exigiram maior configuração inicial e uma organização estrutural mais detalhada, especialmente devido ao uso de linguagens mais robustas como *Java* e *C++*. Entretanto, essas tecnologias proporcionaram maior controle sobre o desenvolvimento do projeto.

4.2.1.2 Arquitetura e Modularização

Durante as implementações, observou-se que a *LibGDX* e a *Raylib* favoreceram uma organização mais modular do código, utilizando separação em múltiplas classes e responsabilidades distintas para entrada do usuário, renderização e lógica do jogo. Em especial, a implementação em *C++* apresentou uma estrutura mais próxima de aplicações de baixo nível, utilizando ponteiros e referências, arquivos de cabeçalho e separação entre declaração e implementação.

A implementação em *Pygame* apresentou uma estrutura mais procedural e linear, reduzindo a complexidade da arquitetura, porém com menor modularização em comparação às demais tecnologias, apesar de ser possível a separação de funções e responsabilidades em diferentes arquivos Python, utilizando importações entre módulos, a flexibilidade da linguagem permite desenvolver sem o uso intensivo de programação orientada a objetos. A *Godot*, por sua vez, apresentou uma abordagem híbrida, utilizando nós e scripts para organizar os componentes do jogo.

4.2.1.3 Curva de Aprendizado

A curva de aprendizado variou significativamente entre as tecnologias analisadas. A *Pygame* apresentou maior facilidade inicial devido à simplicidade da linguagem *Python* e da

própria biblioteca. A *Godot* também demonstrou acessibilidade durante o desenvolvimento, principalmente pela documentação e tutoriais de terceiros.

A *LibGDX* exigiu maior compreensão sobre programação orientada a objetos e estruturação de aplicações em *Java*. Já a implementação utilizando *Raylib* e *C++* apresentou a maior complexidade entre as tecnologias estudadas, devido à necessidade de lidar com conceitos mais avançados da linguagem, como ponteiros, referências e organização em arquivos de cabeçalho.

5 CONCLUSÕES

De forma geral, observou-se que tecnologias com maior nível de abstração, como *Godot* e *Pygame*, proporcionaram maior rapidez de desenvolvimento e menor complexidade inicial. Em contrapartida, tecnologias mais próximas do baixo nível, como *Raylib* com *C++*, exigiram maior conhecimento técnico, mas ofereceram maior controle no desenvolvimento.

É importante salientar que na escolha da tecnologia deve levar em consideração o objetivo do projeto, o nível de experiência do desenvolvedor e os requisitos específicos do jogo a ser desenvolvido. Para projetos mais simples ou para iniciantes, tecnologias como *Pygame* ou *Godot* podem ser mais adequadas. Já para projetos que demandam maior controle e performance, tecnologias como *Raylib* podem ser mais indicadas, desde que o desenvolvedor possua o conhecimento necessário para lidar com a complexidade envolvida.

A experiência de desenvolver múltiplas versões do mesmo jogo permitiu compreender como diferentes tecnologias influenciam diretamente na organização do código, na complexidade do desenvolvimento e na implementação de mecânicas de jogos digitais.

REFERÊNCIAS

ALVES, G. B. **Estudo comparativo entre engines de desenvolvimento de jogos 2D**. Quixadá: [s.n.], 2020. Trabalho de Conclusão de Curso (Graduação em Engenharia de Software) - Universidade Federal do Ceará, Campus de Quixadá. Disponível em: <<http://repositorio.ufc.br/handle/riufc/58827>>. 20, 33, 34

BORGES, L. E. **Python para Desenvolvedores: Aborda Python 3.3**. 5. ed. Sebastopol, CA: O'Reilly Media, 2013. ISBN 978-1449355739. Disponível em: <https://books.google.com.br/books?hl=pt-BR&lr=&id=eZmtBAAQBAJ&oi=fnd&pg=PA5&dq=python&ots=VFQvqsDjdr&sig=AG1HKk0ksn0-NCDD1ifSFVVqz7c&redir_esc=y#v=onepage&q=python&f=false>. 31

BRZUSTOWSKI, J. **Can you win at TETRIS?** Tese (Doutorado) — University of British Columbia, 1992. Disponível em: <<https://open.library.ubc.ca/collections/ubctheses/831/items/1.0079748>>. 38

CODAKID. **The Definitive Guide to Video Game Genres**. 2025. <<https://codakid.com/video-game-genres/>>. Acesso em: 5 set. 2025. 13, 14, 15, 16, 17, 18

CRAWFORD, C. **The Art of Computer Game Design**. [S.l.]: McGraw-Hill / Osborne Media, 1982. Versão eletrônica disponível em: <https://www.digitpress.com/library/books/book_art_of_computer_game_design.pdf>. Acesso em: 4 set. 2025. 12, 13, 16

DEITEL, P.; DEITEL, H. **C++ Como Programar**. 5. ed. São Paulo: Pearson, 2005. ISBN 978-85-352-0490-1. 26, 28

DEITEL, P.; DEITEL, H. **Java: Como Programar**. 8. ed. São Paulo: Pearson, 2016. ISBN 978-85-430-0202-2. 23, 24

DOWNEY, A. **Think Python**. [S.l.]: O'Reilly Media, Inc., 2012. Disponível em: <<https://books.google.com.br/books?id=1mZtP9H6OMQC>>. 30, 31

DYER, R.; CHAUHAN, J. An exploratory study on the predominant programming paradigms in python code. In: **Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering**. ACM, 2022. (ESEC/FSE 22). Disponível em: <<http://dx.doi.org/10.1145/3540250.3549158>>. 31

EGENFELDT-NIELSEN, S.; SMITH, J. H.; TOSCA, S. P. **Understanding Video Games: The Essential Introduction**. 4. ed. New York: Routledge, 2020. ISBN 9780367331978. 9

FIRMANSYAH, R.; ADHITYA, M. The implementation of rpg concept on breakout games using libgdx framework. **Journal of Information Technology**, UIN Sunan Gunung Djati Bandung, 2021. Disponível em: <<https://join.if.uinsgd.ac.id/index.php/join/article/view/517/210>>. 26

Godot Engine Community. **Godot Engine Documentation**. [S.l.], 2025. Acesso em: 16 set. 2025. Disponível em: <<https://docs.godotengine.org/en/stable/>>. 20, 21

- GULARTE, D. **O que é um jogo eletrônico?** 2018. <<https://bojoga.com.br/artigos/dossie-retro/o-que-e-um-jogo-eletronico/>>. Publicado em: 28 maio 2018; Acesso em: 5 set. 2025. 11
- INDRUSIAK, L. S. **Linguagem Java**. [S.l.], 1996. Acesso em: 16 set. 2025. Disponível em: <<https://www.cursosavante.com.br/cursos/curso177/conteudo8707.pdf>>. 22, 23
- LEWIS, I.; BESWICK, S. Generalisation over details: The unsuitability of supervised backpropagation networks for tetris. **Advances in Artificial Neural Systems**, v. 2015, p. 1–8, 04 2015. 38
- LIBGDX. **LibGDX Wiki**. 2025. <<https://libgdx.com/wiki/>>. Acesso em: 16 set. 2025. 25, 26
- LUCCHESI, F.; RIBEIRO, B. Conceituação de jogos digitais. **São Paulo**, v. 7, 2009. 10, 11, 13
- MANOCHA, V. **The Ultimate Guide To Video Game Genres**. 2025. <<https://gameopedia.com/blogs/the-ultimate-guide-to-video-game-genres/>>. Acesso em: 4 set. 2025. 13, 14, 15, 16, 17, 18, 19
- MORAIS, F. C. Desenvolvimento de jogos eletrônicos. **Revista e-xacta**, 2009. Disponível em: <<https://revistas.unibh.br/dcet/article/view/242/134>>. 32, 33
- NEWZOO. **Global Games Market Update Q2 2025**. Newzoo, 2025. Accessed: 30 October 2025. Disponível em: <<https://newzoo.com/resources/blog/global-games-market-update-q2-2025>>. 8
- Oracle Brasil. **O que é Python?** 2025. Acesso em: 19 set. 2025. Disponível em: <<https://www.oracle.com/br/developer/what-is-python-for-developers/#cleancode>>. 30
- PANTOJA, J. I. d. Q. Estudo comparativo de motores de jogos no desenvolvimento de um jogo 2d para web. Quixadá, 2022. Orientador: Prof. Dr. Jefferson de Carvalho Silva; Coorientadora: Profa. Dra. Valéria Lelli Leitão Dantas. Disponível em: <https://repositorio.ufc.br/bitstream/riufc/70844/1/2022_tcc_jiqpantoja.pdf>. 35, 36
- Pygame Community. **Pygame: Python Game Development**. [S.l.], 2025. Acesso em: 19 set. 2025. Disponível em: <<https://www.pygame.org/>>. 32
- SALMELA, T. **Game Development Using the Open-Source Godot Game Engine**. Bachelor's Thesis, 2022. Disponível em: <https://www.theseus.fi/bitstream/handle/10024/746943/Salmela_Tero.pdf>. 21
- SCHUYTEMA, P. **Design de Games: Uma Abordagem Prática**. São Paulo: Cengage Learning, 2008. 447 p. ISBN 9788522106407. 9
- STATISTA. **Video Games - Worldwide | Market Outlook**. 2025. Accessed: 30 October 2025. Disponível em: <<https://www.statista.com/outlook/amo/media/games/worldwide>>. 8
- STROUSTRUP, B. An overview of c++. In: **Proceedings of the 1986 SIGPLAN Workshop on Object-Oriented Programming**. Association for Computing Machinery, 1986. Disponível em: <<https://dl.acm.org/doi/pdf/10.1145/323779.323736>>. 26

TUROS, L. **Introdução à Linguagem de Programação C++**. 2024.
<https://osprogramadores.com/blog/2024/08/24/introducao_a_linguagem_de_programacao_cpp/>. Acesso em: 18 set. 2025. 26

VALLADARES, R. S. **Raylib: A Simple and Easy-to-Use Library to Enjoy Videogames Programming**. [S.l.], 2025. Acesso em: 19 set. 2025. 29

VASQUES, F. C. **Jogar artístico: a experiência do ‘jogar’ como arte**.
Dissertação de Mestrado — Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP), Instituto de Artes, São Paulo, 2025. Acesso aberto; Disponível em:
<<https://hdl.handle.net/11449/296294>>. 9